

From Code to Careers: Linking Open-Source Software Signals to AI Skill Demand in the Labour Market

Dafydd James Thomas

Department of Computer Science
Aberystwyth University
Wales

September
2025

This thesis is submitted in partial fulfilment of the
requirements for the degree of Master of Science

Degree: MSc Data Science

Module: CHM9360

Supervisor: Dr Arina Buzdalova

Abstract

The rapid pace of technological innovation in artificial intelligence (AI) is reshaping labour markets by creating demand for new and evolving skill sets. Traditional labour market forecasting methods, which often rely on static occupational taxonomies and assumption-laden models, struggle to capture the semantic complexity of skill requirements and to anticipate the influence of innovation signals that precede demand shifts. Moreover, the slow-paced nature of building such forecasts is straightforwardly ill-fitted to manage the fast-paced nature of AI disruption. This dissertation addresses this gap by developing a natural language processing (NLP) pipeline designed to extract, represent, and compare skill information across domains: from job advertisements, which reflect market demand, and from open-source repositories on GitHub, which reflect innovation and adoption activity, this represents a real-time method for monitoring the state of labour demand that incorporates novel software innovation data.

The core of the pipeline is a similarity-based model built on sentence transformer embeddings. Unlike keyword-matching systems, which rely on brittle lexical overlap, embedding-based methods provide a semantic representation that enables cross-domain comparison between job ads and GitHub repositories. To evaluate this approach, an ablation study is conducted on alternative pipeline configurations, including variations in preprocessing, representation, and similarity measures. Performance is benchmarked against a primitive keyword-matching baseline, highlighting the relative advantages of embedding-based methods in capturing both direct and contextual references to AI-related skills.

Using this framework, time-series representations of skill demand derived from job advertisements are compared with innovation signals derived from GitHub, operationalized as repository star counts. This exploratory correlation analysis investigates whether activity on open-source platforms may serve as an early indicator of shifts in labour demand. While the analysis remains preliminary and descriptive, it demonstrates the potential of linking innovation and labour market signals through a unified semantic representation.

The dissertation makes three contributions. First, it introduces a modular and reproducible NLP pipeline for cross-domain representation of skills, suitable for both benchmarking and extension. Second, it provides an empirical evaluation of embedding-based models for labour market skill extraction, demonstrating clear performance improvements over keyword-based baselines. Third, it explores the integration of innovation-derived signals into labour market forecasting, offering initial evidence of correlations between open-source activity and demand for AI-related skills.

Together, these findings suggest that embedding-based NLP pipelines provide a stronger foundation for analysing evolving skill demand and that innovation signals from open-source ecosystems may play a valuable role in anticipating future labour market needs. This work thus contributes both methodological tools and exploratory insights to the growing field of computational labour market forecasting.

Declaration of originality

I confirm that:

- This submission is my own work, except where clearly indicated.
- I understand that there are severe penalties for Unacceptable Academic Practice, which can lead to loss of marks or even the withholding of a degree.
- I have read the regulations on Unacceptable Academic Practice from the University's Academic Quality and Records Office (AQRO) and the relevant sections of the current Student Handbook of the Department of Computer Science.
- In submitting this work, I understand and agree to abide by the University's regulations governing these issues.

Name: Dafydd James Thomas

Date: 05/09/2025

Consent to share this work

- By including my name below, I hereby agree to this thesis being made available to other students and academic staff of the Department of Computer Science, Aberystwyth University.

Name: Dafydd James Thomas

Date: 05/09/2025

Acknowledgment

To whoever has the patience to read this :-)

This section is customary, but not obligatory. It makes a brief statement of thanks to those who have helped.

Contents

1	Introduction	1
1.1	Aims & Objectives	2
1.1.1	Primary Aim	2
1.1.2	Objectives	2
1.2	Roadmap	3
2	Literature review	4
2.1	Economic Background	4
2.1.1	Labour (Employment) Demand & Dynamics	4
2.1.2	Theories of Innovation	4
2.1.3	Role of Artificial Intelligence	5
2.2	Labour Demand Forecasting	6
2.2.1	Overview	6
2.2.2	Typological Approaches to Forecasting	6
2.2.3	Alternative approaches	11
2.2.4	Key Insights	12
2.3	Open Source Software (OSS)	13
2.3.1	Open-source concept	13
2.3.2	Open-source for Software	13
2.3.3	OSS Importance	14
2.3.4	OSS as Innovation	14
2.3.5	Quantifying OSS innovation	15
2.3.6	Key Insights	17
2.4	Natural Language Processing	18
2.4.1	History	18
2.4.2	BERT and Sentence Embeddings	18
2.4.3	Natural Language in Job Postings	19
2.4.4	Natural Language in GitHub repository	21
2.5	Synthesis and Research Gap	22
2.5.1	Integrated Understanding	22
2.5.2	Justification of Current Approach	23
3	Methodology Overview	25
4	Data	28
4.1	Introduction	28
4.2	job postings	28
4.2.1	Overview of Data Landscape	28

4.2.2	Problems in Acquisition	29
4.2.3	Solution: Kaggle	29
4.2.4	Cleaning & Collating	30
4.3	OSS repositories	32
4.3.1	Overview of Data Landscape	32
4.3.2	Repository selection	32
4.3.3	Acquisition: Web Scraping	33
4.3.4	Acquisition: GitHub's REST API	33
4.3.5	Acquisition: GitHub's GraphQL API	34
4.3.6	Results	34
4.4	Auxiliary Resources	34
4.4.1	Keyword Dictionary	34
4.4.2	ESCO Skill Classification	35
5	Embedding Representation	36
5.1	Background	36
5.2	Sentence Embeddings	37
5.3	Similarity Computation	38
5.4	Evaluating Similarity	38
6	Baseline (Keyword-based) Model	40
6.1	Motivation	40
6.2	Specification	41
6.3	Keyword Dictionary Design	42
6.4	Baseline intersection matrix	43
7	Experimentation	45
7.1	Text Preprocessing	45
7.1.1	v1	45
7.1.2	v2	46
7.2	Sentence Embedding Models	46
7.3	Dimensionality Reduction	46
8	Evaluation	48
8.1	Introduction	48
8.2	Evaluation metrics	49
8.3	Evaluation results	52
8.4	Discussion	52
9	Cross-domain Correlation Analysis	54
9.1	Introduction	54
9.2	Analysis	54
9.3	Discussion	55
9.3.1	Mean values	55
9.3.2	Correlations	58
9.3.3	Broader Trends	60
9.4	Summary	61

10 Critical Evaluation	64
10.1 Project-Level Problems	64
10.1.1 Research Goal & Directions	64
10.1.2 Code organization	64
10.2 Strengths	65
10.3 Assumptions	65
10.3.1 GitHub repositories as proxies for OSS	65
10.3.2 Job postings as proxies for labour demand	66
10.3.3 Job Posting Source concatenation	66
10.3.4 Job Title Relevance	67
10.3.5 Using <i>OSS Insight</i> Collections	68
10.4 Limitations	68
10.4.1 Data Limitations	68
10.4.2 Relying on General-Purpose Sentence Transformers	68
10.4.3 Baseline Model Design	69
10.5 Opportunities for future work	70
11 Conclusion	71
Appendix A	81
A.1 Generative AI	81
A.1.1 Elicit	81
A.1.2 ChatGPT	82
A.1.3 Cursor	83

List of Figures

2.1	The inter-connected Economy	5
3.1	Conceptual diagram of methodology pipeline.	26
4.1	Distribution of job postings over time, in quarterly periods - we have specific concentrations (most notably at 2024Q1), 2 Q's with minimal data 2 with nonetheless	30
4.2	Distribution of Job Titles for job postings - <i>Data Scientists</i> dominate and few of the more directly AI-related (<i>MLOps</i>) are low. Rather than being a sample-issue, this is likely reflecting an inherent imbalance between demand for either role.	31
6.1	Confusion Matrix: A binary classification model is either correct (TP / TN) or incorrect (FP / FN). Optimizing the model means minimizing the <i>incorrect</i> judgements, but there exists a trade-off between FPs & FNs: Reducing one type of error often increases the other.	42
9.1	Lag Correlation example: cos & sin are ideal examples to demonstrate the notion of lagging one sequence to find different lag-correlation values with another	56
9.2	Scatterplot of mean-popularity vs. mean-similarity: most repositories have low mean-popularity, with a fairly broad mean-similarity	57
9.3	(Part 1) Boxplot of repository similarity values, ordered by mean similarity: This shows the range of similarity values that each repository expresses.	58
9.4	(Part 2) Boxplot of repository similarity values, ordered by mean similarity: This shows the range of similarity values that each repository expresses.	59
9.5	Distribution of correlation values (Similarity vs. Popularity): An even distribution of correlations, more random than meaningful	60
9.6	Distribution of correlation values (Similarity vs. Popularity) at respective <i>best lags</i> : Most repositories have a high-level of correlation when factoring in time-lags	61
9.7	Distribution of correlation values (Similarity vs. Popularity) at respective <i>best lags</i> , coloured by repository age	62
9.8	Age of Repositories: The distribution demonstrates that a burst of related repositories are created $\approx 10 - 12$ quarters ago (between 2 and 3 years - roughly in the 1 st and 2 nd quarters of 2023)	63

9.9 Aggregate trends: Orange bars indicate number of repositories created at that time period; Green bars indicate the number of repositories that peak in quarterly popularity; blue lines indicate the mean similarity across all repositories, with the standard deviation . 63

List of Tables

4.1	Overview of datasets used for Job Postings	30
7.1	Specifications of three sentence embedding models.	46
7.2	Strengths and trade-offs of three sentence embedding models.	47
8.1	Evaluation of experiments: the various configurations (runs) of the sentence-embedding inputs & outputs are evaluated on various metrics. The v2_c run appears strongest.	52

Chapter 1

Introduction

In recent times, we are seeing that skills and job requirements are evolving rapidly. As the culmination of various effects, including the development of new technologies and evolving consumer preferences, the labour force must adapt in order to satisfy the new requirements. Given the historic pace of labour demand shifts, traditional methods of forecasting these changes are not well-equipped to handle this blistering, and accelerating, rate of change.

By these so-called "Traditional" methods, I mean that which accompany government or firm-level analysis of the skills market. These depend upon data which are limited in the following ways:

1. They use structured data
2. They collect the data periodically (e.g. Annually / Quarterly)

An important feature of these methods is their approach to modelling the dynamics influencing labour demand. Of key importance is the impact of innovation, and how new technology is estimated to affect skill requirements.

This paper aims to investigate whether Open Source Software (OSS) innovation Signals, derived from unstructured text data, can meaningfully contribute to the forecast of AI-related skill requirements present in job postings.

To break this down, there are a few key ideas being explored:

1. Can open source software data represent information about Innovation in the AI sector?
2. Can AI-related skill requirements be modelled the textual data within job postings?
3. Can we integrate the two data channels to produce a quantified measure of their relationships?

Put simply, if we can measure how similar an OSS repository (innovation-source) is to job postings, in terms of the skill-profile and concepts involved, then we can represent how this innovation-source has diffused over time (i.e. by the date of job posting). By relating this to the temporal development of the innovation-source, as represented by some other measure of the OSS repository’s history, we can the development-diffusion timeline of that particular innovation.

If true, this insight has tangible benefits for the practice forecasting skill demand in the AI sector, within which open-source software is a prolific source of innovation. Moreover, job postings & OSS data are high-frequency and accessible, meaning that more granular and fine-tuned predictions can be produced in effective real-time.

1.1 Aims & Objectives

1.1.1 Primary Aim

To develop and evaluate an AI-related skill demand forecasting methodology, integrating innovation signals (from Open-Source Software in particular) and using job ads as a proxy for skill demand.

1.1.2 Objectives

1. **Data Collection:** Collect & collate data for both channels in:
 - (a) Open-source Software (GitHub): *textual content & historic measure of development*
 - (b) job postings (e.g. LinkedIn, Indeed) - *content & date-posted*
2. **Representation:** Develop a high-quality representation method for integrating textual data from both channels, involving:
 - (a) **Build a Baseline:** A keyword-based mechanism for baseline matching of text corpora.
 - (b) **Advanced Representation:** An intelligent and robust method for representing text as embeddings.
 - (c) **Experimentation:** Fine tune and alter the components of embedding model, evaluating against the baseline.
3. **Time series:** Investigate Relationship of derived time series, looking at the dynamics of correlations at lagged time periods.

4. **Discuss:** Critically evaluate the derived results, with reflections on the benefits and limitations of the approach.

1.2 Roadmap

In the following chapter, I discuss related pieces of work and how their findings informed the paradigm for this project's implementation. Given the domain intersectional nature of this paper, works that are "related" include those on the topics of economic theory, natural language processing and AI itself.

Following this, the implementation of this project is detailed. Firstly, this concerns how data was acquired and preprocessed. Thereafter, the primary component of the pipeline is discussed - namely, the representation pipeline. Within this, I demonstrate both how the baseline model and the representation model are built, discussing the involved components and how experimentation can be conducted to find optimal constructions. The subsequent section demonstrates the downstream task of these representation results - in the context of time series analysis, the findings of our representation pipeline are presented alongside the data of GitHub Stars. Having demonstrated the function of the entire pipeline, this next section therefore details the results and rationale of several sets of experiments.

What thereafter follows is a critical evaluation of the process, discussing the project's limitations and its successes, as well as the opportunities for future work. Finally, We discuss in the conclusion the outcomes and results of this paper, and their implications.

Chapter 2

Literature review

2.1 Economic Background

2.1.1 Labour (Employment) Demand & Dynamics

Labour demand is defined as the number of people that employers are willing to employ at a given wage during a specific time period.

As a result of various economic phenomena, Labour demand is dynamic and frequently changes: Firm-level performance, competition and policies in minimum-wage or taxation are among the many factors that can influence a firm's capacity to employ. At the microeconomic (firm/individual) scale, much of the negative effects of labour demand equilibria changes Are expected in a free-market economy.

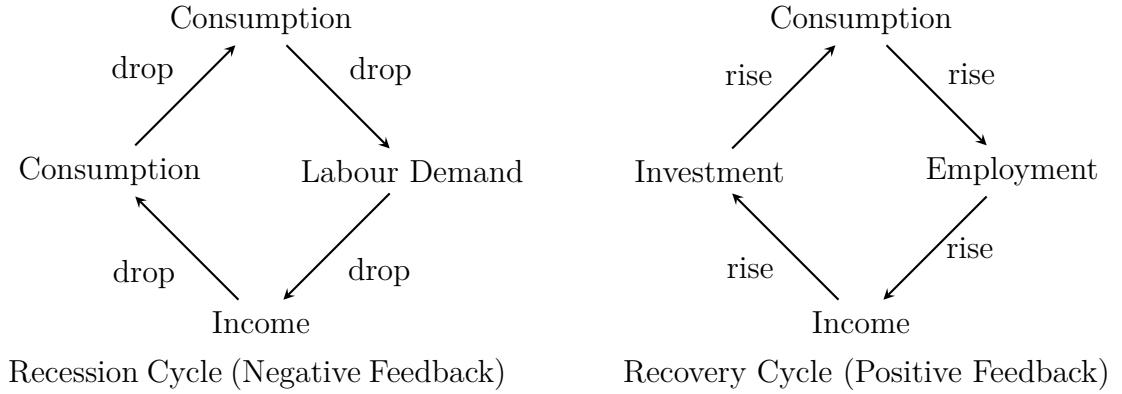
There are, however, systematic shocks that would affect the labour demand on a macroeconomic (economy-wide) sense. These can be deeply damaging to the proper functioning of that economy. Examples of such systematic shocks include war, pandemics or financial crises. Broadly, these shocks will have effects that extend to beyond labour demand shifts, since the economy is composed of an unfathomably complex network of cause-effect relationships.

Such a consequence of this interconnectedness is the existence of feedback loops within the economy, as the standard Keynesian representation of recessions depicts [1] (see Figure 2.1):

2.1.2 Theories of Innovation

Another driver of shifts in Labour demand is technology, or more generally, as innovation. Classical economic theory and some research suggests that technological development & adoption can be seen as a labour augmentation device, contributing positively to long term growth and employment [1], [2]. An opposing view, most famously represented by the example of the Luddites [3], depicts

Figure 2.1: The inter-connected Economy



technological advancement as detrimental to employment prospects.

The theories of innovation are a historically rich field of research. One guiding theory is the *Combinatorial view of Innovation*, originally envisioned by Schumpeter [4]. This is the view that much of innovation derives from combining aspects of existing knowledge in novel ways, and it is empirically justified in a wide variety of scenarios.

For example, the widely cited work [5] examines the nature of innovative research papers, and finds “The highest-impact science is primarily grounded in exceptionally conventional combinations of prior work yet simultaneously features an intrusion of unusual combinations”.

Rogers’ (1983) *Diffusion of Innovations* is another theory of innovation focuses on the factors that determine how diffusive an innovation is - diffusion in this case is the “process by which an innovation is communicated through certain channels over a period of time among members of a particular culture” [6]. Rogers depicts that there are four elements affecting the spread of new ideas, which model the transition as resulting from social and network factors. The wide-adoption of an innovation is integral to its sustenance on this view, but its key insight is that social networks hold the key to understanding whether an innovation will prevail.

There are many other theories of innovation. The authors of [7] assess and classify the meta-theories of technological innovation from classical texts and modern sources. Among their findings is that there are a wide variety of views on the dynamics of technological innovation, each exhibiting different causal understandings as depicted by the authors’ meta-theory classifications.

2.1.3 Role of Artificial Intelligence

Artificial intelligence is perhaps in a separate category of technological innovation for a few reasons. As is its nature, it is a different type of technology as it is

inherently regarded as *intelligence*. This can be seen as both complementary and substitutional to human intelligence or labour.

This gives rise to emergent phenomena as it relates to technological innovation and labour:

1. **AI *as* innovation:** AI as a technology in the classical efficiency multiplier sense.
2. **AI *drives* innovation:** The authors of [8] examine the effects that AI has *on other* technological innovation, finding a strong positive relationship, especially in low-tech sectors.
3. **AI *replaces* innovation:** Artificial General Intelligence (AGI) is the abstract concept of a technology with an intelligence surpassing that of all humanity.

It is the 3rd point here that is of deep importance, because it represents such a profound shift a civilizational sense. Aside from the controversy and disagreement that surrounds the topic of AGI, it is clear that Labour demand will be profoundly impacted by the continual development of AI.

2.2 Labour Demand Forecasting

2.2.1 Overview

The practice of forecasting the demand (and *supply*) for labour has a long history. Many critique its utility [9], but there is good reason to believe that the limitations faced by current approaches are not insurmountable. What follows is the findings of research into existing approaches, with an emphasis on their limitations.

2.2.2 Typological Approaches to Forecasting

2.2.2.1 Manpower Requirements Approach [9]

The authors of [9] discuss the *Manpower Requirements Approach (MRA)* for forecasting labour demand, within the context of Aboriginal Canadian communities. For context, the MRA approach is still central to the US's Bureau of Labour Statistics (BLS) occupational-forecasting methodology, among other major institutions. In summary, the approach utilizes a “future macroeconomic reference scenario for GDP [...] to derive an estimate of total employment demand, which is later broken down by industry and by occupation.”

The authors modify the approach slightly, due to some concerns about feasibly producing “macroeconomic reference” scenarios as they concern the Aboriginal population of Canada. As a result, alongside a separate model they define for labour supply, their methodology is considered to be a fairly comprehensive framework for understanding demand & supply.

An important contribution they make is to lament the importance of forecasting not only “Expansion” demand (an increase in demand in absolute terms) but also “*Replacement*” demand (an increase in demand resulting from gaps created by mortality, retirements, occupational mobility, etc.).

However, the methodology is deeply assumption-led and is consequently limited.

- **Static occupational coefficients:** They assume that occupational distribution by industry remains constant over time
- **Zero elasticity of substitution between labour types:** workers in different occupations aren’t easily substitutable.
- **Extrapolation of trends:** e.g., labour force participation or enrolment rates follow past patterns—even if future social or structural shifts may alter them significantly.

Furthermore, as the authors recognize, the framework is fraught with other issues including:

- Data Limitation problems for smaller institutions
- An assumed heterogeneity of individuals within occupations misses a granular understanding of the individuals’ differences (e.g. their skill-sets)

To add to data limitation problems, the authors find that a best practice is to use “up-to-date data and regularly refresh forecasts (every 2–3 years, medium-term horizon of 5–10 years)”. This is insufficient with respect to today’s economy, especially considering the fast-paced shift in the labour market as a result of AI.

This review of labour forecasting methods is not without its issues, but highlights some important considerations for future endeavours.

The major limitations of their approach are:

1. Lacking granularity beyond occupation-level
2. The assumptions are highly simplified
3. Data requirements are extensive
4. Time-wise granularity is poor

Moreover, occupation-level findings of such forecasts are increasingly irrelevant, for reasons detailed later.

2.2.2.2 Quantitative Econometric Approaches [10]

A still prevalent and perhaps predominant approach to demand-side forecasting is through econometric macroeconomic models: linking macro indicators (like GNP or industry-level output) to employment and occupational demand. These are often regarded as theoretically sound because of how they respect the underlying economic principles regarding individual behaviour. To a large extent, these models are supremely effective in modelling large-scale phenomena resulting from the microscale (e.g. individual/household level). A contributor to their successful usage is their capacity to express causality resulting from a vast array of factors (wages, capital, trade, productivity, technology, etc.). In any model of a real-world system, simplifying assumptions are necessarily present and as such, the major concern is that they do not represent the real-world sufficiently well - it is debatable that this even matters if a model is predictively powerful [11].

But there are major limitations to solely depending on econometric models:

- **Time Horizon:** Though purporting to estimate with long horizons, they often break down after the medium-term.
- **Data problems:** Expressive econometric models require long-running detailed time-series of various data factors. This is demanding even for economies with developed data infrastructure, and simply unfeasible for those with less developed data infrastructures.
- **Limits to understanding:** across time, parameters may evolve to be more/less relevant, or their relationships alter fundamentally - this change is not accounted for by the top-down specification of the model assumptions.
- **Fallacy of past behaviour being indicative:** problematic in the face of widespread technological disruption as is expected with AI.
- **Technical (Statistical) difficulties:** often, highly technical models are not useful unless they are highly interpretable, which remains difficult for non-technical policymakers or employers.
- **Inherently unpredictable events:** the models often suffer / break-down during periods of economic shock (e.g. financial crises, technological shifts).

In practice, such models are often best used in combination with other approaches, such as survey-based or other empirically grounded techniques. Furthermore, they discuss and address explicitly that though approaches like MRA utilize an occupation-level granularity for forecasts, the lost nuance is detrimental to their defensibility. They recognize that policy-makers and governments will find

two/three-dimensional forecasts (relating to industry/occupation/qualification dimensions) increasingly irrelevant and incapable of producing informative forecasts in the face of those utilizing multidimensional data (“functions, tasks, abilities, personal attributes, characteristics and generic skills”).

Data Limitations and detailed understanding are the primary concerns that the authors raise as to the viability of econometric forecasts.

2.2.2.3 Survey-based Approaches [12]

Another common method of gauging labour demand is through employer or establishment surveys. These typically involve asking firms directly about current vacancies, expected future hiring, or perceived skills shortages. They can be tailored to particular industries or occupations, offering a more immediate and micro-level picture of demand.

The strength of survey-based approaches is that they can capture employer expectations in near real-time, and may reveal demand for skills that are not well reflected in historical data or official occupational classifications. They are also relatively straightforward to implement and can provide insights into qualitative aspects of demand (e.g. “soft skills” requirements).

However, their limitations are notable:

- **Short time horizons:** employer expectations rarely extend beyond 1–2 years, limiting their usefulness for medium- or long-term forecasting.
- **Sampling and response bias:** firms willing to respond may not be representative, and self-reported expectations are subject to optimism or pessimism.
- **Lack of standardization:** surveys are often ad hoc, making results difficult to compare across sectors or countries.

These issues mean that survey-based forecasts are best viewed as complementary, rather than substitutes, for model-based approaches.

2.2.2.4 Input–Output Approaches [10], [13]

Input–Output (I–O) approaches use inter-industry transaction tables to model the relationship between output in one sector and the demand for inputs (including labour) in others. By combining I–O tables with projected sectoral growth, forecasts can be generated for labour demand by industry and, through further assumptions, by occupation.

This approach has the advantage of providing a structured, economy-wide framework that captures interdependencies between sectors. It is particularly useful for analysing the employment effects of shocks or structural changes in specific industries.

However, several limitations arise:

- **Static technical coefficients:** I–O models generally assume fixed relationships between inputs and outputs, ignoring substitution and technological change.
- **Data intensity:** compiling detailed and up-to-date I–O tables is resource-heavy and not always feasible for smaller economies or specialized subgroups.
- **Poor skill granularity:** I–O models focus primarily on industry demand, meaning occupational and skill-level forecasts require additional strong assumptions.

Consequently, while valuable as part of a comprehensive forecasting toolkit, I–O approaches suffer from the same rigidity and assumption-dependence as the manpower requirements and econometric methods discussed earlier.

2.2.2.5 UK Context

Robert A. Wilson, the author of [10], contributes also to many specific forecasting efforts especially considering the UK government’s Working Futures programme (WFP). This is the primary tool that the UK government uses projecting future employment demand, and is very comprehensive.

The Working Futures Report [14], produced by the Institute for Employment Research (IER) of Warwick University [15] in partnership with other institutions (including the UK Commission for Employment and Skills (UKCES) and the Department for Education), is a “comprehensive, systematic, consistent and transparent set of projections” of the likely future’s labour market. Their projections are centrally grounded by projections of the E3ME Cambridge Econometric Model [16], producing multi-sectoral & multi-regional projections for 75 industries. Their rationale suggests that “sectors / industries [are] the key driver of changes in employment by occupation” [14]. Producing 10 year projections at intervals of 3 years, the report also makes use of with modules for replacement demand, occupations, and qualifications, with data from the Office for National Statistics (ONS) [17]. Framed not as exact predictions, the purpose of this forecast is to inform policy given outlooks or views of potential futures.

An immediately noticeable limitation of this approach is the periodicity of updates. For example the 2017 – 2027 forecast is produced in 2020. With a base

year of 2017, the forecast can only be produced after a lag-time for the purpose of acquiring the relevant data, such as national figures, which are released at a lag of 1 – 2 years. Thus, the next report, grounded in 2020, would be expected to release in 2023 - this most recent report is structured somewhat differently, with a longer time-horizon (2020 – 2035) [18].

To some extent, the report is immune to a critique of its long-term accuracy, given its framing as a “future likelihood” tool. Nevertheless, *improved* accuracy is always tangibly beneficial. Those critiques (as the report itself details) discuss those limitations:

- The forecast is heavily assumption-led
- The forecast results are not designed for precise local-area forecasts
- The forecast may lack the nuance of *intra*-occupational change

Further, a critique produced by [19] finds that many forecasts share the same limitations:

- Difficulty in linking from macroeconomic trends or employment forecasts to skills
- Few attempt to understand how skills change *within* occupations.

2.2.3 Alternative approaches

There are a number of solutions that are being investigated that aim to resolve some of the aforementioned limitations of labour market forecasting.

In the past, “naturally occurring data” (data not produced for analytical purposes) such as online job postings were difficult to assess systematically because of their lack of structure [20]. Furthermore, it has not historically been the case that vacancies are always advertised online, thereby adversely affecting their strength in representing labour market demand. However, these concerns are both increasingly irrelevant, due to advancements in Machine Learning techniques and the almost ubiquitous practice of advertising vacancies online.

These offer a potentially powerful data-source for understanding how the labour market is changing, and are under thorough investigation in many situations. One example is how the author of [21] uses Job Posting text to predict salaries, demonstrating the informative and predictive power of the information expressed by the natural text source.

Among those most pertinent to this paper are those that extract and portray the evolution of skill-requirements.

The Bank of England (BoE) employ an unsupervised Machine Learning (ML) techniques with job-posting data to identify disaggregation/segmentation methods for forecasts that are not at a sector/occupation level as in the WF Report, but are instead at a skills-level [22]. They recognize the power of this Machine Learning based approach to devise data-driven taxonomies for skills that are otherwise unobserved by official classifications, which “cut across wage, sector and occupation”.

The European Centre for the Development of Vocational Training (CEDEFOP) have an ongoing project in their *Skills Online Vacancy Analysis Tool for Europe* (Skills OVATE), which uses vast data pipelines from 27 Nations (EU member states, EEA countries and the UK) to produce *quarterly* (every 3 months) updates on a skill-demand forecast [23]. Another example of how skills can be investigated using job postings is presented by the OECD. They analyse Job-posting to produce an outlook on the demand for AI-related skills across sectors [24].

Other research suggests the power that Job Postings have to produce meaningful insights into how skills within occupations are changing [25]–[27].

2.2.4 Key Insights

The approaches available for a given economic actor to produce forecasts of labour market demand are varied. Whilst mostly different in their these methods, there are shared problems.

In general, for comprehensive models, high quality & voluminous data are needed at regular intervals, from a panoply of sources. This is deeply problematic not only for those private individuals with few resources available to them, but also to firms and even policy-makers, since acquiring such data introduces a necessarily longer lag-time for new forecasts. Furthermore, macroeconomic models experience difficulty in projecting intra-occupational change - this is fatal for properly understanding a labour market which is driven fundamentally by the more granular unit of individual skills, from which occupation-level requirements are composed.

As such, a variety of solutions are being investigated that take a more data-driven and bottom-up approach for understanding this more functional disaggregation of labour-demand - i.e. in terms of *skills*. Integrating the information observed in naturally-occurring data such as job postings to aid forecasts is ongoing, but they nevertheless show promise due to the informative, high-frequency and high-volume nature of job posting data, and are therefore worthy of consideration.

2.3 Open Source Software (OSS)

2.3.1 Open-source concept

Open-sourcing is a powerful concept within technological information. Before the internet, published information was centrally controlled, by the publishers or curators of such information. With the conception of internet tech, widespread collaboration on such information sources became viable, leading to openly available and extensively developed sources of information such as Wikipedia - this version of an encyclopedia is different to its paper-printed ancestors because *anyone* has the ability to propose and implement changes (under administrative audit). As a result, Wikipedia is the go-to encyclopedic source of information, known for its vast and detailed coverage of topics, as well as its reliability in presenting up-to-date and trustworthy information.

2.3.2 Open-source for Software

Open-source software (OSS) is the release of software under a licence that permits (under varying conditions) the software's modification and redistribution [28]. On the whole, OSS is designed to promote collaboration and innovation within software, and is a paradigm under which many of the most powerful software tools are developed. Analogous to what Wikipedia is for general information, the most well known "host" of OSS is GitHub - this is a platform which provides extensive access to, and methods to collaborate with (or contribute to), OSS projects. There are other such platforms, but GitHub is by far the most popular. Central to what makes GitHub successful is its powerful version-control system (Git) - functionally, this is the mechanism by which multiple people can work on a document simultaneously without conflicting changes, as well as providing routes to revert changes and control permissions.

For the remainder of this paper, I shall use interchangeably the notions of OSS and GitHub:

Stipulation

The terms **GitHub (GH)** and **Open-Source Software (OSS) Platform** (and equivalently, **GH repositories** and **OSS products/projects**) are used interchangeably.

Though GitHub is not the *only* source-code host for OSS, it is by far the most popular [29]. Furthermore, given that the scope of this discussion is not that of GitHub (the platform) and instead concerns its hosted projects (i.e. repositories), this stipulation is relevant because I intend to use GH repositories as proxies for

OSS projects in general (The defensibility of this proxy assumption is examined under Section 10.3.1). This is an important distinction, and promotes pragmatic/effective discussion without unnecessary jargon.

2.3.3 OSS Importance

Many view that open-source is a practice that underpins the success and advancement of software technology. As the authors of [30] report, OSS should be treated as a public good. They further lament the unseen cost to developers within OSS, as well as specifying the necessity of upkeep for such ecosystems.

Indeed, given the expectations transformative power of AI, it is regarded open-sourcing relevant software is more important than ever [31], [32]. It is seen as vital that powerful software systems such as AI ought to be democratized to prevent exploitation arising from private ownership, allowing the benefit to distribute more evenly among the population: “We believe the open source approach is the only way to achieve the full potential of AI, making it safer, accessible and democratized” [31].

2.3.4 OSS as Innovation

Open source software (OSS) is increasingly a rich ecosystem for The development of new software technologies. Often it is regarded as a hub for experimentation and innovation, and is a unique depiction of innovation outside the incentive structure observed by private economic agents.

Weak ties (low-intensity, infrequent interactions) are a long-theorized and recognized driver of innovation, since in a broad sense, they expose individuals to ideas from unrelated economic & social circles [7]. The authors of [33] investigate the impact of weak-ties, in the context of software development circles, on software innovation. Their findings suggest that a developer/contributor’s future project innovativeness is strongly correlated with the breadth and diversity of projects they engage with. This is highly suggestive of the power that the OSS ecosystem has to induce a positive externality-like effect for software innovation.

Similarly, the authors [34] examine “the imports of libraries in 12 different programming language ecosystems within millions of Stack Overflow posts over a 15-year period”. They aim to present learnings of OSS innovation to the literature on innovation dynamics, and find that recombination is a key innovation process in software (resembling concepts such as combinatorial innovation [4]). This is novel because they demonstrate the efficacy of treating the OSS ecosystem as similar to patenting and scientific research in terms of the innovation dynamics they express. The authors of [35] observe similar dynamical traits, doing so with

a novel approach of examining the dependency growth and complexity of GitHub repository.

2.3.5 Quantifying OSS innovation

A key consideration for understanding the impact or proliferation of an OSS project, thereby in attempt to measure its innovative qualities, is defining how this is *measured*. GitHub, being designed for maximally efficient & transparent collaboration with optimal version control, naturally offers openly available metrics for projects as relating to their development. These include *forks*, *commits*, *contributors* and so on. Within each there will be detail regarding the specific instance, the time and their specific natures.

For example, a commit will include information on what files have been changed and a commit message describing those changes. A *contributor* is a user who has, at least once, made a commit to a project - a contributor-project association graph can thus be defined. A *fork* is a duplication of a project in its current state to a separate *branch*, useful for refactoring or adding new features without affecting the working (production) source code. When satisfied with the outcome of a fork, a merge is created to update the main branch.

These explanations miss a little nuance, but it remains clear that there are several avenues for tracking and monitoring the development of a GitHub repository.

One other metric, that is somewhat seen separately to those naturally occurring data regarding a repository’s development, are “GitHub stars”. The literature regarding stars suggests a lack of clarity as to their primary purpose, mostly because there isn’t a functional one as regards development.

A major study into the usage and impact/relevance of stars is conducted by the authors of [36]. In earlier work they investigate the popularity growth of GitHub repository [37]. They “[identify] four main patterns of popularity growth, which are derived after clustering the time series representing the number of stars of 2,279 popular GitHub repository”. Among their findings is that stars & forks are strongly correlated, but Stars & contributors and stars & commits are weakly correlated. Additionally, they find that repositories with high number of stars are overwhelmingly owned by companies or institutions, rather than individuals. A key insight of theirs is to suggest that many repositories see a burst of star-gains after their initial release, which almost always drop off immediately after - this suggests that bursts of popularity are not indicative of long run popularity growth.

A common theme of their research is to compare the popularity of GitHub repository to similar studies in social media popularity phenomena, such as classifying popularity growth patterns: Slow, Moderate, Fast, and Viral.

Expanding in later work [36], they:

1. Expand their analysis to 5000 repositories,
2. Survey 791 developers to identify the motivation / reasons behind starring,
3. Investigate of factors influencing star-growth (e.g. marketing),
4. Develop a Machine Learning method to identify specific factors that predict a GH repository’s popularity growth.

Among their primary findings is that “Stars are a key metric about the evolution of GitHub projects”, and as such their monitoring is important for project managers. They identify that, in addition, what prompts *fast* star growth is often the result of “active promotion, usage of trending technologies and innovative projects”. However, *virality* is presented as a caveat to the general treatment of stars as a repository’s popularity: often, this is the result of marketing and social media promotion. As such, the high star-count is often a result of a burst of stars gained during certain periods. They therefore urge caution when selecting GitHub projects by number of stars, and to check whether these stars were gained over sustained periods or within short time-frames.

In [38], the relationship of a repository’s stars and its downloads are investigated. In summary, their results find that both are “indicators for the importance of client-side JavaScript libraries”. Whilst not immediately relevant to all types of GitHub projects, such as those in other programming languages (e.g. Python) or for other end-goals (such as for AI or ML), the results are nonetheless promising in support of Stars as a metric for project importance.

The authors of [39], building on the research supported theoretical concept of combinatorial innovation, examine the trends and relationships of such recombination projects in the Python programming language. Their results suggest that higher levels of innovativeness are strongly correlated with high-star counts - “Novelty begets popularity”. Interestingly, they also note that the relationship between contributor-count and innovativeness is inversely correlated.

An interesting approach followed by [40] attempts to build a *predictor* of repository stars given other features of a repository, using a few Machine Learning models. The features they use fall within categories of:

- **General Project Characteristics:** locality, commits per developer, number of forks, number of pull requests (PRs), total contributors, repository age, programming language (and more)
- **Characteristics of Developers and organization:** followers per developer, developers per organization (and more)

- **Activity:** commits per day, ratio of accepted/total PRs

Their best results arose from using PCR and Random Forest predictors, with R^2 (explained variance in the dependent - i.e. stars - as a result of the independent variables) values of 0.87 and 0.88 respectively. With an analysis on how the mean squared error (MSE) of the predictions change with the removal of features, they find that the most important features are:

- Number of forks
- Number of followers per organization member
- Number of PRs
- Number Average number of commits per day
- Maximum number of followers per organization member

The most prominent of these (forks) is strongly supported elsewhere, but otherwise, the “follower” trait of an organization member is a surprising and novel result.

This suggests that popularity is observable not only in stars, nor only in forks, but also in the network characteristics of a repository’s contributor community.

2.3.6 Key Insights

Within software, OSS is much more relevant in understanding innovation than are other innovation signals (such as research papers or patenting). Whilst there is perhaps strong evidence to suggest that multiple metrics would form better appreciation for the innovativeness and utility of a GitHub repository, there is strong support for treating stars as a proxy measure for popularity. By extension, since popularity is a good indicator of novelty, stars are a simplistic but often effective measure of a repository’s innovativeness but with caveats: the timeline of stars is of equal importance, since bursts of star gains are not indicative of much more than strong promotional and/or marketing efforts. Instead, long sustained star-growth trends are more indicative of innovativeness. Moreover, it is worth noting that under Rogers’ [6] theory of innovation, it is diffusion through networks that results in the overall success of an idea - this should be seen as a supporting principle of stars as a measure of innovation diffusion, at least within the software community. Furthermore, the weak-ties investigation [33] is somewhat analogous to [5], and supports this idea that OSS ecosystems exhibits Schumpeterian tenets too. It’s nice to see (unrelated) classic economic theories have something informative to say about OSS ecosystem’s innovation system.

2.4 Natural Language Processing

2.4.1 History

The simplest Natural Language Processing (NLP) technique is keyword matching: querying a body of text to determine the presence of a defined keyword. Though straightforward, This method is too narrow to capture the complexity of *natural* language, which encompasses alternative terms of phrase, acronyms and so on. To be useful, this method would involve curating a large dictionary of terminology, which is laborious.

An improvement would be to use fuzzy matching techniques, which allows for matching on typos and some small patterns alternative phrasing. However, this approach remains limited in its ability to capture *meaning*. It is not enough to simply define a match on a word-for-word basis, because a crucial element of what makes natural language meaningful is the context. For example the word “Python” alone means something very different when seen in the context of “Python, cobra and anaconda” as compared to “Python, SQL and Java”.

A major breakthrough in NLP came with the shift from sparse, dictionary-based features (like standalone words or phrases) to *dense vector embeddings*, where words and phrases are represented as points in a continuous high-dimensional space. Early methods such as Word2Vec [41] and GloVe [42] captured semantic similarity by training embeddings such that words occurring in similar contexts are placed near one another.

The subsequent development of *self-attention* mechanisms, formalized in the Transformer architecture as depicted by the groundbreaking *Attention is all you need* paper [43], extended this idea by dynamically weighting the relevance of surrounding tokens when generating a representation. This allowed embeddings to become *contextual*: the same word, such as “Python”, receives different vector representations depending on its sentence context (programming vs. reptile). Together, distributed embeddings and self-attention form the foundation for modern models like BERT, enabling scalable and context-sensitive language understanding.

2.4.2 BERT and Sentence Embeddings

BERT (Bidirectional Encoder Representations from Transformers) [44] is trained using a masked language modelling (MLM) objective (in which tokens in a sentence are randomly masked, and the model must predict them using the surrounding context) and a next sentence prediction (NSP) objective (which trains the model to understand sentence-to-sentence relationships). Together, these tasks

yield rich token-level representations that are *context-dependent*: the embedding of “Python” differs depending on whether it appears alongside other programming languages or reptiles.

In practice, BERT encodes each token in a sentence as a dense vector in a high-dimensional space. Because of the bidirectional self-attention mechanism, these token embeddings capture both syntactic and semantic relationships with their surrounding context. Downstream NLP systems can fine-tune BERT’s parameters, or simply use its representations as input features, to achieve state-of-the-art performance on a variety of tasks (e.g. question answering, named entity recognition, or classification).

However, BERT is primarily designed to produce *token-level embeddings*. When a sentence/document-level representation is required, further processing is needed. A common approach is to average the embeddings of all tokens, or to use the special [CLS] token, which is trained to aggregate sentence information. Later models, such as Sentence-BERT (SBERT) [45], build directly on BERT to optimize for high-quality sentence embeddings by introducing a contrastive training objective. This allows semantically similar sentences (e.g. “She enjoys programming in Python” and “She likes coding in Python”) to be mapped to nearby points in the embedding space, while dissimilar sentences are mapped further apart. Sentence embeddings thus form the basis of semantic similarity search, clustering, and retrieval-augmented methods that are critical to modern NLP pipelines.

2.4.3 Natural Language in Job Postings

2.4.3.1 Skill and Competence Taxonomies

To represent the “skill profile” of a Job Posting (JP) or a GitHub repository (GHR), we need a **taxonomy** to define the concepts of a “skill”. Official classification bodies such as ESCO and O*NET exist for this broad purpose. They define hierarchical classifications to depict skills requirements for occupations.

The term *skill* itself is, however, only narrowly defined by the *European Skills, Competences, Qualifications and Occupations* (ESCO) classification [46] as “the ability to apply knowledge and use know-how to complete tasks and solve problems.” ESCO further distinguishes *knowledge* as “the outcome of the assimilation of information through learning,” encompassing facts, principles, theories, and practices related to a field of work or study, and defines *competences* as “the proven ability to use knowledge, skills, and personal, social, and/or methodological abilities in work or study situations and in professional and personal development.” Thus, competences capture the broader capacity to apply both knowledge and skills within specific contexts.

These classifications are problematic for the purposes of this project, because in practice, ESCO does not make a clear distinction between skills and competences within its “skills pillar”. ESCO also defines many items that are ability-wise relevant to AI (e.g. specific programming languages) as knowledge rather than skills. Additionally, ESCO classifications are manually defined, and may therefore be missing the nuance of recent & emergent taxonomies in the domain of AI. The strict and somewhat outdated ESCO classifications are therefore problematic for framing what the “skills” in this domain are.

Therefore, for the remainder of this paper, I shall stipulate the following:

Stipulation

The term “skill” is in this paper used broadly to encompass all three ESCO categories insofar as they relate specifically to AI technology.

This serves to simplify the discussion of how GitHub and Job Postings relate within the domain of AI technology, and should therefore not be confused with how ESCO defines a skill.

2.4.3.2 NLP for Skill-extraction

In [47], the authors develop a data driven methodology to extract and represent a job-skill taxonomy. Emphasizing that they do so independently of official boards for taxonomy definition (e.g. O*NET and ESCO), they Leverage NLP and network analysis methods for an algorithmic methodology to construct skill taxonomies, which remain adaptable and avoid expert elicitation. The significance of this is that it reinforces the feasibility of using NLP techniques, and specifically sentence embeddings, to extract skills from job postings.

Named Entity Recognition (NER) is a method of Natural Language Processing which identifies and extracts known entities (such as companies, people, or locations) from a body of text. The authors of [48] develop a specialized annotated dataset of 301 Job Postings (JPs) that serves to validate NER models for skills & knowledge (treated separately) extraction. Their annotations extend beyond the basic token-wise annotation as regularly done with NER models, and instead use *span* labels (using BIO (Beginning, Inside, Outside) tagging). This is done because often, skill-entities are extended spans of text (e.g. "Ability to debug code").

Their process takes 2 baseline models, *BERT* and *SpanBERT*. They utilize a set of unlabelled 3.2M Job Postings to domain-adaptively pre-train these two models to produce a further 2 models, which they call *JobBERT* and *JobSpanBERT*.

With their annotated dataset, they fine-tune each model by passing sentences

through the model’s transformer encoder, through a linear layer, after which the output of a Conditional Random Field (CRF) (“framework for building probabilistic models to segment and label sequence data” [49]) layer returns BIO-tags for the sentence. The models are trained to maximize sequence-level likelihood under the CRF.

In essence, their research contributes:

1. A novel dataset of labelled Job Postings with span-level annotations: span-level annotations are useful because they are more layered than basic token-level annotation. E.g. They allow for nested tagging: “ability to use Python programming libraries” - as a whole span of text, this is seen (as under ESCO classification) as a *skill*, whereas Python is itself considered *knowledge*. Both aspects are tagged within their specialized dataset.
2. Evaluate the performance of baseline vs. domain-adapted BERT and SpanBERT models on this annotated dataset.

They find that their domain-adaptive pre-trained models outperform baselines, with JobSpanBERT performing best on Skills and JobBERT performing best on knowledge. The results of this paper are such that there is purportedly clear benefits to adaptively training a BERT, and hence transformer-derived, models on domain-specific data, but this is less true regarding knowledge components: a difference of 1.5 F1 for the BERT and JobBERT on knowledge.

A limitation of this paper is the small dataset of annotated examples by which they use to evaluate their models’ performances. As is often the case, data annotation remains a barrier to extensive and nuanced models capable of generalized high-performance in this case.

2.4.4 Natural Language in GitHub repository

A growing body of work demonstrates that the natural language fields of GitHub repository—such as titles, descriptions, README files, and user-defined topics—encode rich signals about their technical scope and knowledge content.

Repo2Vec [50] combines textual metadata with structural and code-based information to create repository embeddings, showing that natural language fields alone were sufficient for effective similarity search, clustering, and even malware detection.

Another function is to utilize NLP methods for building *topic recommendation* systems. A feature of GitHub repository is the option to tag a project with relevant topics - these are useful for navigating between and grouping similar repositories. However, repository owners do not always tag their projects sufficiently, and often

not at all. Solutions to this treat repositories as multi-label text classification problems, concatenating descriptions, READMEs, and other textual components, and applying both traditional document representations (TF-IDF, Doc2Vec) and contextual embeddings from Transformer models such as DistilBERT [51]. These methods achieve strong performance in predicting repository tags and demonstrate that semantic representations of repository text align well with human-curated technical taxonomies.

Other work extends this principle by linking repositories to external knowledge domains. For example, *paper2repo* have been developed to match academic papers with relevant repositories by embedding both corpora in a shared semantic space [52]. The success of this method is important in that it justifies, to some extent, the viability using semantic embeddings for comparing the similarity of textual content *across domains*.

Graph-based approaches such as HiGitClass [53] similarly leverage README and description text as node features in heterogeneous networks, showing that textual content improves repository classification across languages and domains. More recent efforts use encoder-only LLMs, such as BERT, DistilBERT and RoBERTa, to produce Transformer-based sentence embeddings that classify README sections into functional categories with high accuracy [54], further underlining the representational value of repository documentation.

Taken together, this line of research supports the view that GitHub repository can be modelled as vectors of their textual features. These embeddings capture the *technical concepts* (e.g. programming languages, libraries, frameworks) and *knowledge concepts* (e.g. problem domains, tasks, skills) embedded in repository descriptions and documentation, making them a valuable feature source for downstream analyses of technological innovation and labour demand.

2.5 Synthesis and Research Gap

2.5.1 Integrated Understanding

On the whole, then, this extensive literature phase has informed the following takeaways:

1. Comprehensive labour demand forecasting methods are time-intensive, data-heavy and struggle to capture nuances of skill-level changes. They are laden with assumptions and are not designed in a manner that expects underlying relationships between factors to change. Given the fast-paced and expected highly disruptive nature of AI development, there is great need for

time-effective and data-driven forecasting methods that are adaptable and relevant.

2. Job postings present an avenue for such data-driven forecasts. Given their nature as high-frequency and high-volume data-points, they can reliably be treated as proxies for real-time labour demand. While historically they have been unfeasible to mine and represent in an analytically useful manner, new methods of NLP overcome this limitation.
3. OSS and GitHub repository are widely viewed as key to innovation in software technology, much of which AI and AI-related competency is connected to. The popularity of such OSS projects on GitHub (namely, repositories) can be, to some degree, measured by a proxy-measure of their “stars”. This is an increasingly recognized manner in which the conceptual themes of technological can be assessed: by monitoring the stable, and continued, growth of stars in a repository.
4. GitHub repository can be represented as a feature of their textual content, breaking into topical/technical classifications that extend beyond their explicitly defined topics. Furthermore, there is some evidence to suggest the validity of producing cross-domain embeddings that can be assessed in a semantic context, such as via similarity. This is supportive of an approach to represent GitHub repository in terms of features that are shared with other textual corpora (e.g. Academic papers, as shown, and potentially Job Postings).

There is not, as I am aware, a concerted research effort being conducted into assessing the **spillover** of innovation from OSS $\rightarrow$ Labour Demand. This is a major gap in current research and needs investigating.

2.5.2 Justification of Current Approach

The methodology of this paper therefore aims to explore this gap in understanding. With my understanding of the *Diffusion of Innovations*, those that have pervasive and sustained impact through social circles are the innovations with the highest impact. Therefore, it stands to reason that such strong innovations in *software* (whose most innovative catalyst is that of OSS ecosystems) are resultant in later demand for *knowledge* of that software.

This cross-domain understanding is then assessed based on the quantifiable existence of:

1. Temporal Job Posting as proxy labour demand

2. Temporal GitHub repository stars as popularity/diffusion

Additionally, NLP methods are supportive of:

1. Representing Job Postings as expressive of skills
2. Representing GitHub repository text in terms of technical details

This research aims to develop a cross-domain representation pipeline for the textual elements of Job Postings & GitHub repository, thereafter utilizing their respective temporal modalities that are useful for answering the diffusion question.

Chapter 3

Methodology Overview

The purpose of this chapter is to more concretely depict the stages of the pipeline detail what is involved, step by step - each are then in turn described in detail over the following chapters. My usage of the word “pipeline” is inspired by the nature of the implementation being a full, end-to-end solution (as opposed to specific instances or model developments). A crucial part of what makes such a pipeline successful is a reliable and operational *flow* of computational tasks.

In a brief sense, we can suggest that there are three main stages to this pipeline, all of which are in terms of underlying source(s) of data:

1. **Acquisition:** finding appropriate data, and preparing it for “processing” readiness.
2. **Processing:** Otherwise called “representation”, this step involves transformations of the acquired data to become analytically useful. In ML, for example, this is the training of a model (using the acquired data) to produce meaningful outputs (predictions/classification) that may thereafter be used for analytical purposes.
3. **Analytics:** Investigation of underlying data, as represented by the intermediate form, with regards to the initial research question.

Whilst vague, this can help guide expectations for the various steps in between, since with long and complex pipelines it can be difficult to assert the full relevance of each stage in isolation. A version of this pipeline, therefore, is represented neatly by the diagram in Figure 3.1.

Specifically then, if the goal is to measure diffusion of OSS innovation into labor demand, then We want to measure the relationship between two temporal series:

1. The trend of *innovation* - The evolution & popularity on the development side,

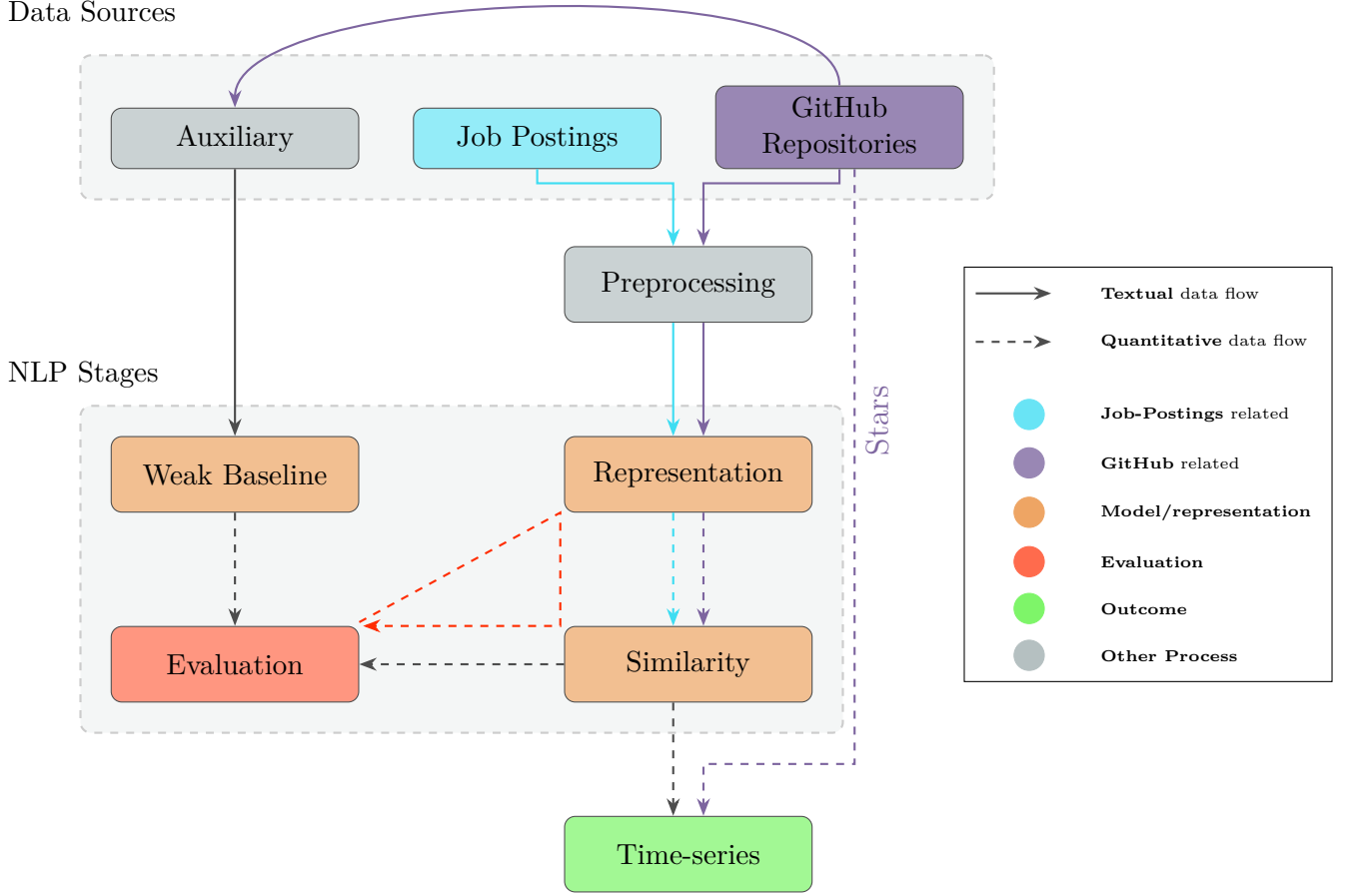


Figure 3.1: Conceptual diagram of methodology pipeline.

2. and the trend of *adoption* - The requirement of possessing the relevant knowledge/skills.

The first of these is straightforwardly representable by the so-called supply side of development (specifically, GitHub) metrics. The second, however, is challenging: there is no straightforward way to depict whether a repository is relevant to a job and vice versa.

Therefore, along the way, we must construct a method to represent this relevance factor. As the literature suggests, **Sentence Embeddings** present a clean solution - in an intra-domain sense, the method is demonstrably viable for evaluating the semantic similarity of documents. In addition, there is evidence to suggest that *cross-domain* semantic similarity can be assessed using sentence embeddings, though there is no research currently that supports this idea in this context (job postings vs. GitHub repository).

The bulk of this paper is therefore dedicated to establishing a **high quality representation of cross-domain semantic similarity**. The following describes each component of the pipeline in detail:

1. Data collection,
2. ETL & EDA (Pre-processing, Exploring & Cleaning),
3. Baseline model development (Fuzzy-matching keywords),
4. Sentence embedding representation,
5. Evaluations and experimentations (Textual Variations, Sentence-transformer variation, Dimensionality Reduction),
6. and Alignment & Analysis (Correlation, lead-lag) of resulting time-series'.

Chapter 4

Data

4.1 Introduction

This phase of the project establishes a systemized workflow to find and pull data. This effort can be grouped in three distinct ways categories (or channels):

1. job postings,
2. GitHub repository,
3. and Auxiliary data.

4.2 job postings

4.2.1 Overview of Data Landscape

As previously discussed, the practice of online advertisement of employment vacancies is a ubiquitous in recent times. In one sense, this is a benefit because concerns about existence of data are not pertinent. A panoply of websites, colloquially known as “job boards”, exist online - the primary examples of these include *LinkedIn* [55] and *Indeed* [56], though these sites are much more than vacancy-advert sites.

Some job boards have dedicated API layers. For example, among the various available LinkedIn API endpoints (under paid access) is one that allows for efficient programmatic interfacing to find vacancies in occupational-fields or specific roles - this is useful for both employers and employees alike. In other cases, web-scraping is a viable option, albeit more laborious than using APIs.

4.2.2 Problems in Acquisition

However, a severe limitation of acquiring data in real-time from these sites is a result of the fleeting nature of job postings: Once a vacancy is filled, the advert is no longer necessary, so the data-point is no longer maintained on the job board site. Because of this fact, directly accessing job posting data from job boards means retrieving that data as it appears (and before it disappears) in real time.

Given that the scope of this paper investigates the long-term relationship between data-sources (diffusion can be a slow process), it is not enough to *begin* collecting data now.

Rather, it is necessary to acquire pre-existing datasets consisting of historic records. This proved problematic, because largely, such data is expensive to store, and as such is not considered by job-board sites as necessary to maintain. Moreover, because of this economic cost of storing this data, the most comprehensive access to such data is expensive: Firms such as Lightcast [57] are framed as “labour market intelligence” firms that offer data & analytics services, pertaining to job postings under the labor market data.

4.2.3 Solution: Kaggle

Thankfully, the open data community of Kaggle [58] proves mightily useful for such data needs. This is a platform for Data Science and Machine Learning where people share lessons, data & code for education and competitive purposes. As a resource, it is often the solution to data acquisition: in most cases of niche requirements, someone has likely created a relevant dataset on Kaggle.

It turns out that this is true for job postings, and emphatically so - there exist hundreds of such datasets. Not all data sets are of equal value - many are very small and many have limited features - And as such I located and downloaded 5 adequately expressive datasets. The details of these datasets are expressed in Table 4.1.

As can be seen, some datasets are created by individuals through web-scraping methods, while others are sample datasets from the much larger stores of commercial entities (e.g. Techmap[65]). Each of these data sets satisfy the minimal criteria for utility in this project. Ranging from a few hundred to a few thousand records, they all possess the required features:

- Date component: A timestamp of when the posting was created.
- Title component: The advertised job title.
- Description component: The main body of texts for a job posting.

Table 4.1: Overview of datasets used for Job Postings

Dataset Name	Date Range	Details	Source
1000 ML Jobs US	12/2023 - 04/2025	Mostly within March & April of 2025, all jobs are ML related	[59]
Linkedin Data Engineer Job Postings	17/12/2023 17/12/2023	- Extra "skills" column - This is appended to "description"	[60]
Data Science Job Postings & Skills (2024)	19/01/2024 21/01/2024	- Extra "skills" column - This is appended to "description"	[61]
LinkedIn Job Postings (2023 - 2024)	23/03/2024 20/04/2024	- -	[62]
Job Postings from Ireland (October 2022)	01/10/2022 31/10/2022	- Ireland only; Sample dataset from Techmap.io's commercial product	[63]
US Job Postings from 2023-05-05	05/05/2023 05/05/2023	- USA Only; Sample dataset from Techmap.io's commercial product	[64]

4.2.4 Cleaning & Collating

These five datasets cover different temporal (See Fig 4.1), regional and occupational (See Fig 4.2) ranges. This next stage involved reconfiguring their composition into a single representative dataset.

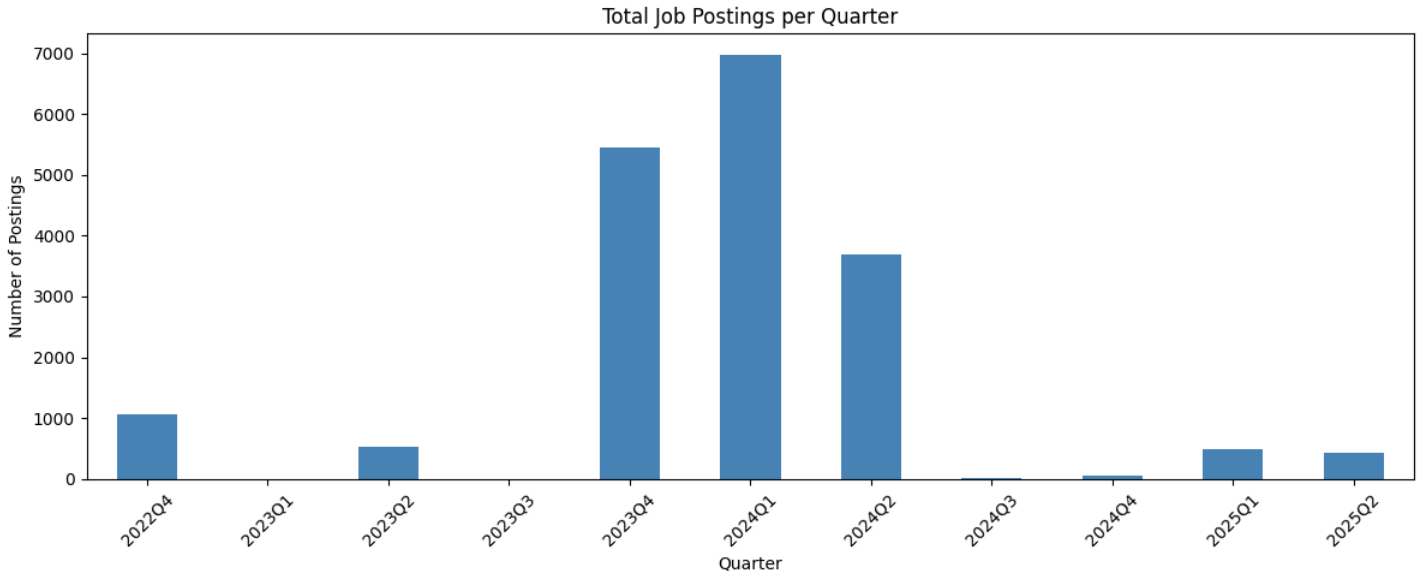


Figure 4.1: Distribution of job postings over time, in quarterly periods - we have specific concentrations (most notably at 2024Q1), 2 Q's with minimal data 2 with nonetheless

As is the case for the majority of this project, this process was undertaken using

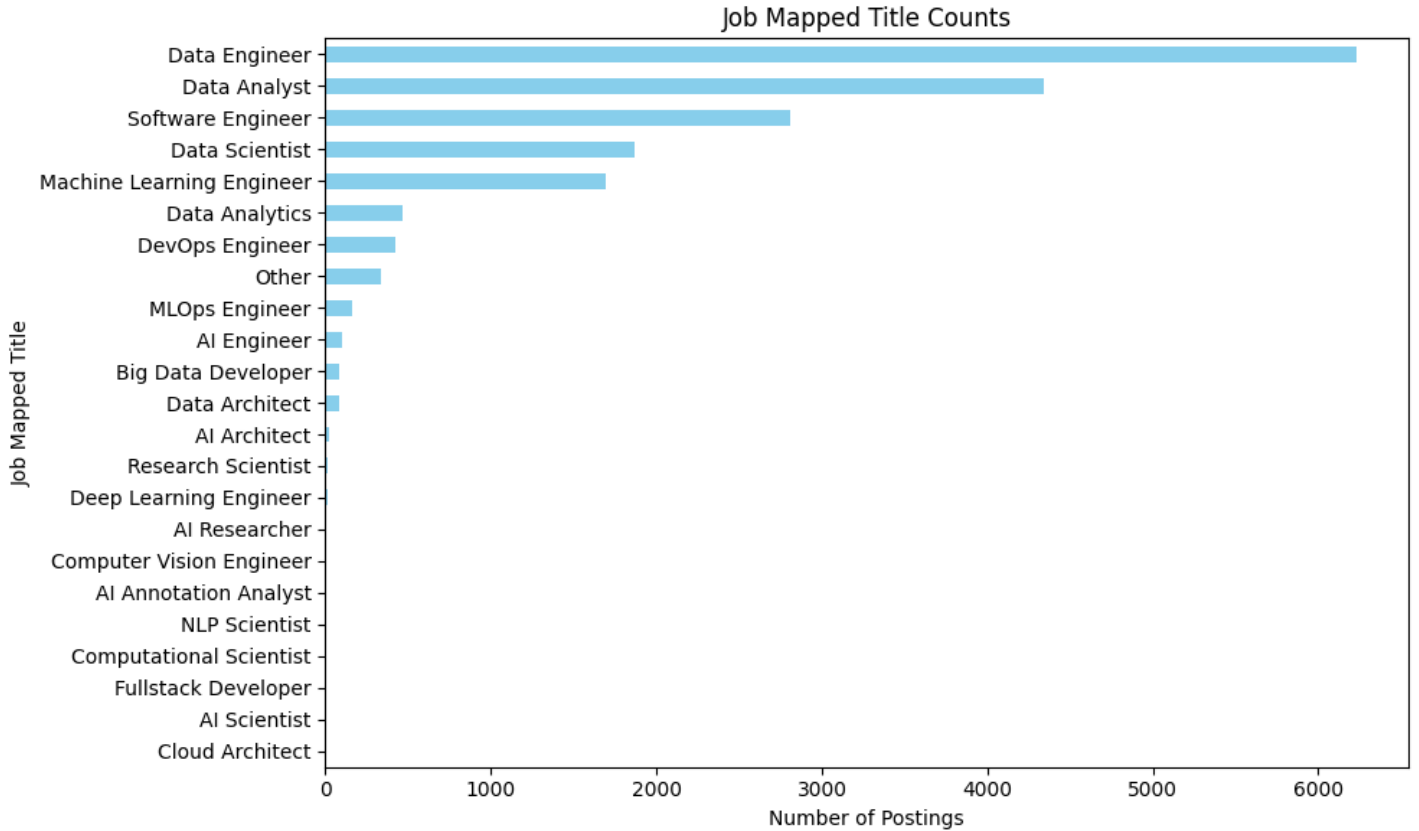


Figure 4.2: Distribution of Job Titles for job postings - *Data Scientists* dominate and few of the more directly AI-related (*MLOps*) are low. Rather than being a sample-issue, this is likely reflecting an inherent imbalance between demand for either role.

Python (and the library **Pandas** - full information on utilized software & libraries is under Appendix A.1.3.2). Some datasets included an additional “Skills” field. During pre-processing, I concatenated this to the “Description” field so that all job postings are represented as a single textual body. This ensures consistent treatment across postings, whether or not they explicitly separate skills from the main description.

Thereafter, each dataset required varying degrees of processing to retrieve & represent the 3 necessary fields, which are thereafter compiled into one big **Pandas DataFrame**. Thereafter, I generated a new field - “Mapped Title” - which uses *regular expressions* to map the noisy “Title” field into a cleaner aggregation field. Using this, undesirable fields (i.e. not relating to AI or Data Science) were removed - this was done in alignment to the design choice as set out in section 10.3, and the method is largely effective (assuming the proper **regex** definitions).

As a result, the full job postings dataset for downstream usage totals 18690 records ranging from 10/2022 → 04/2025.

4.3 OSS repositories

4.3.1 Overview of Data Landscape

The process for retrieving GitHub repositories data is much simpler in the availability sense - free API access is offered, via 2 alternative endpoints: REST API and GraphQL API. To reiterate, the features of this data channel required are the repository metadata (including the README, description, owner-name, repository-name, and topic-tags) and popularity data, representable by count of stargazers (a ‘stargazer’ of a repository is a user that has starred that repository) as a timeline of the respective creation-events (When the user ‘starred’ the repository). It is important to note that these are but a subset of the available data points that are collectible, and usable, from GitHub.

For example, it is possible that other forms of textual data exist on an ad-hoc basis, potentially providing more fine-grained technical information that is relevant to representing its ideas. Furthermore, other measures of popularity and of development evolution Would provide a more holistic understanding of a repository’s true development history. Nonetheless, the choices made are pragmatic in that they are, on the whole, sufficiently indicative of repository characteristics, for any repository.

4.3.2 Repository selection

A subset of repositories was required, as acquiring and processing all of GitHub is infeasible. Manual curation would be both laborious and subject to bias, so automated alternatives were sought.

I approach this in two different ways:

1. Web Scraping by GitHub topic
2. Informed by a Third-Party source

The first approach used seed topics (e.g. Machine Learning”, Natural Language Processing”), retrieving the top 30 repositories per topic and recursively expanding via their associated tags. This method effectively captures a broad subspace of related repositories, but it has drawbacks: niche or irrelevant topics may be included, and many relevant repositories lack topic tags altogether (this latter problem is exactly that as tackled with [51]).

The second approach therefore seeds from an auxiliary data source a list of repositories that are trending/relevant to sub-topics of software engineering / AI. This source, *OSS Insight* [66], is an analytics project that provides various functionalities, including daily-updated topical collections of repositories that are *trending*

or *popular*. Their algorithm for determining these repositories include ‘scores’ of stars/forks and a ‘base score’, computing optimized ratings from weighted treatment of such metrics. Their ranking algorithm is based on a composite score including stars, forks, and a base score, with weights optimized over these metrics. The exact details are not fully disclosed, though their open development process suggests that additional engagement signals may be incorporated (e.g. activity around pull requests and issues, and code evolution in line deletions/creations) [67]. Moreover, their GitHub repository reference that ‘collections’ are curated by a *community process*, wherein users can submit collections in pull requests. This constitutes a form of community-driven annotation, providing an effective low-time/effort alternative to developing a bespoke classification mechanism or manually labeling categories.

These repositories are externally validated as relevant to AI, reducing the need for subjective judgment on my part. All that is left to me is to select the names of “Collections” (analogous to the “seed topics” in my first approach) for usage, as seen in table ?? . Further consideration is paid to the problems and limitations of this approach are detailed in section 10.3. For the purposes of manageability, this is an appropriate method of sampling a relevant set of repositories, and fits nicely into a programmatic pipeline by the simple API access provided.

Importantly, *OSS Insight* metadata (e.g. ‘Collection names’) is not used in downstream analyses, since it references data external to the repository contents. They are perhaps useful for categorization in an evaluative sense, but as they reference data extraneous to the repository contents itself, it would not be appropriate to treat this in kind.

4.3.3 Acquisition: Web Scraping

Initially, I chose to develop a simple web scraping method for pulling repository metadata. Though somewhat effective. This method presents major maintainability issues and furthermore is not necessarily the most compliant mechanism for acquiring website data.

4.3.4 Acquisition: GitHub’s REST API

Thereafter, I graduated to develop a script for acquiring the metadata of GitHub repositories via GitHub’s REST API endpoint. This method is not, however, extensible to observing the star-history for a project. Such data is most effectively stored (and accessed) using a Graph Database, relating as it does to the activity of users, affording the benefit of also storing other forms data inherent to the interconnected network nature of the GitHub (and broadly, OSS) ecosystem.

4.3.5 Acquisition: GitHub’s GraphQL API

Therefore, I developed a method to access GitHub’s GraphQL API endpoint, using defined graph-query-language queries that can effectively be managed/alterd in a separate part of the codebase. This method involves paginating through the set of sometimes hundreds of thousands of star-creation events for a repository, extracting the related time-stamps. At $\approx 1.5 \frac{\text{Seconds}}{\text{Request}}$ & retrieving 100 star-events per request, this process is time-intensive for large repositories (The largest in this project is **TensorFlow**’s repository, with $\approx 190,000$ stars).

To consolidate this process, the metadata acquisition script was refactored to also make use the GraphQL API. This results in a more robust, maintainable and adaptable component for the acquisition of GitHub repository data, which remains extensible to acquiring other data-points (e.g. Forks, Commits, Pull Requests, etc.).

4.3.6 Results

A full dataset of GitHub repositories and their related metadata & stargazer-history are therefore retrieved via maintainable and adaptable code. The script thereafter aggregates and stores stargazer history in different formats, namely wide and long formats:

- **Long:** A tabular file with fields of [`‘date’`, `‘repository’`, `‘stars’`].
- **Wide:** A tabular file resembling a matrix of shape (n, m) , where n is the number of repositories and m is the number of discrete time periods (daily, monthly, or quarterly). The time axis is defined as the range from the earliest star-event across all repositories ($\min D$) to the latest ($\max D$), partitioned into equal-length periods. Each entry (i, t) records the number of star-events for repository i in period t .

4.4 Auxiliary Resources

The term ‘Auxiliary’ here represents a channel of data that is both an assortment of sources that were used for evaluative (or otherwise informative) purposes and not central to the representation/analysis itself.

4.4.1 Keyword Dictionary

As is made clear in section 6.2, there is a need for some ‘ground truth’ measure against which representation methods can be evaluated.

In essence, there is a requirement for such abstract representation methods for *labelled* data that is not easily devised. This would entail definitively creating a metric of “similarity” between job posting/repository pairs (r_i, j_t) ¹, a task that is difficult to define clearly, as well as being heavily time-intensive and laborious. To overcome this issue, a heuristic based method was developed that utilizes a basic *dictionary* of relevant terms/phrases for keyword matching. The curation of this data is integral to the methodology of the evaluation baseline (hereafter referred to as the **Baseline Model**), so the details are left to section 6.2.

4.4.2 ESCO Skill Classification

The full ESCO Skills & Competences classification dataset was downloaded and used for exploring the contents of job postings.

This data is not utilized in any meaningful sense by the project’s pipeline - this is a methodological alignment with philosophy such as in [47], expressing a preference to rely on a data-driven & discovered taxonomy. While there is conceivable utility to the ESCO classification, it is not immediately clear that the sort of shared semantics between job postings and GitHub repositories would resemble their taxonomy, so their contents are left aside for this paper’s primary developments.

¹To aid notation and simpler discussion, an ambiguous pair of (job posting/repository) is (r_i, j_t) , where r_i and j_t refer to the i^{th} and t^{th} (occasionally, t is used to indicate job-index to avoid confusion between the usage of j and t - where there is no such confusion, j is used instead) repository and job posting data-points respectively. The notation is made more clear in section 5.4

Chapter 5

Embedding Representation

5.1 Background

In machine learning, a **representation** refers to a way of encoding abstract or unstructured data into a form that can be quantitatively analysed. In natural language processing (NLP), this typically means mapping linguistic units into numerical vectors.

The most common unit is the **token** (which may be a whole word, subword, or even punctuation). A **dense word vector** (or **embeddings**) representation assigns each token a point in a high-dimensional vector space,

$$w_i \in \mathbb{R}^k,$$

where k is the embedding dimension (often in the hundreds).

The geometry of this space captures semantic and syntactic relationships. For example, linear directions can encode consistent transformations in meaning. A classic illustration is the analogy:

$$\text{King} - \text{Man} + \text{Woman} \approx \text{Queen}.$$

Formally, if $v(\cdot)$ denotes the embedding function, then

$$v(\text{king}) - v(\text{man}) + v(\text{woman}) \approx v(\text{queen}),$$

where “ $\approx$ ” indicates that the resulting vector is close in *cosine similarity* to the target embedding. This shows that semantic relations (such as gender or royalty) can emerge as approximately linear subspaces within the embedding space.

However, in multi-token sequences of text, meaning is often derived from their **interactions and composition** so in isolation, static token-embeddings aren’t representative of contextual meaning. For example, “river bank” and “bank loan”

have the same *token-embedding* for “bank” but they refer to different things, so it is not appropriate that they share representations in the semantic-space.

Transformer encoders are deep learning architectures that enrich each of a text passage’s embeddings with *contextual* information derived from the entire passage. Each token’s vector is iteratively refined through layers of *self-attention*, allowing information to flow between all tokens in the passage. This is profound because it means, for example, that $v(\text{bank})$ is *augmented*, capturing how its meaning changes when it succeeds “river”, or precedes “bank”.

Formally, the contextual embedding of token t_i is

$$h_i = f_\theta(t_1, \dots, t_n)_i, \quad (5.1)$$

where f_θ is the transformer encoder, and h_i encodes both the identity of t_i and the influence of its context.

5.2 Sentence Embeddings

Sentence embeddings extend the idea of embeddings from tokens to whole passages. Instead of producing one vector per token, we want a **single dense vector** that summarizes the semantics of an entire sentence or paragraph. This is achieved by

1. passing the text through a transformer to obtain contextual token embeddings $(h_1, \dots, h_n)$
2. applying a **pooling operation**

Formally,

$$\mathbf{e} = g(h_1, \dots, h_n) \in \mathbb{R}^k, \quad (5.2)$$

where g is a valid pooling operation, such as mean or max.

The result is a **fixed-size embedding** ($\mathbf{e}$) that preserves semantic similarity in vector space: two sentences with similar meaning will be close under cosine similarity, even if they share few surface words. This makes sentence embeddings powerful for tasks like semantic search, clustering, and entailment detection.

Training high-quality sentence embeddings requires a vast amount of data (often on the order of billions of examples) and substantial computational resources, which are beyond the scope of this study. Pre-trained models, by contrast, have been extensively trained and validated on large, diverse datasets, and are widely regarded as robust, general-purpose encoders. Their embeddings have become de

facto standards in tasks such as semantic similarity, retrieval, and clustering. By leveraging these models, this research can focus on the downstream methodological questions—such as how embeddings interact with forecasting frameworks—while standing on a reliable and reproducible representational foundation. As detailed in section 7, three instances of **pre-trained** sentence models are used in this project (Tables 7.1 & 7.2).

5.3 Similarity Computation

Each sentence or passage is represented by a dense vector embedding $\mathbf{e} \in \mathbb{R}^k$ in a k -dimensional semantic space. If the sentence model captures meaningful semantic features, then vectors that are close in direction correspond to semantically similar text passages. This geometric intuition is formalized using *cosine similarity*, which measures the angle between two embedding vectors and therefore quantifies their semantic similarity.

Formally, the cosine similarity s of two $\{\mathbf{a}, \mathbf{b}\} \in \mathbb{R}^k$ vectors is:

$$s(\mathbf{a}, \mathbf{b}) = \cos \theta = \frac{\langle \mathbf{a}, \mathbf{b} \rangle}{\|\mathbf{a}\| \|\mathbf{b}\|} = \frac{\sum_i a_i b_i}{\sqrt{\sum_i a_i^2} \sqrt{\sum_i b_i^2}} \in [-1, 1]. \quad (5.3)$$

Where $\langle \mathbf{a}, \mathbf{b} \rangle$ is the dot product of $\mathbf{a}$ and $\mathbf{b}$. Vectors are often normalized so that $\|\mathbf{x}_{\text{norm}}\| = 1$:

$$\mathbf{x}_{\text{norm}} = \frac{\mathbf{x}}{\|\mathbf{x}\|} \quad (5.4)$$

reducing cosine similarity to a simple dot product:

$$s(\mathbf{a}, \mathbf{b}) = \langle \mathbf{a}_{\text{norm}}, \mathbf{b}_{\text{norm}} \rangle = \sum_i^k \mathbf{a}_i \mathbf{b}_i \quad (5.5)$$

5.4 Evaluating Similarity

The computation of similarities follows by using **normalized** sentence-embedding vectors for repositories and job postings, $\{\mathbf{r}_1, \dots, \mathbf{r}_n\}$ and $\{\mathbf{j}_1, \dots, \mathbf{j}_m\}$, organized row-wise into matrices $\mathbf{R} \in \mathbb{R}^{n \times k}$ and $\mathbf{J} \in \mathbb{R}^{m \times k}$.

The pairwise cosine similarity between repositories and job postings is naturally represented by the similarity matrix

$$\mathbf{S} = \mathbf{R} \mathbf{J}^\top \in \mathbb{R}^{n \times m}, \quad (5.6)$$

with entries

$$\mathbf{S}_{i,t} = s(\mathbf{R}_i, \mathbf{J}_t) = \langle \mathbf{r}_i, \mathbf{b}_t \rangle$$

Since each job posting is timestamped, we can sort the rows of $\mathbf{J}$ chronologically, allowing each row of $\mathbf{S}$ to be interpreted as a time series of similarity values between a given repository and all job postings.

To reduce sparsity and obtain more interpretable dynamics, we aggregate similarities over fixed time intervals. Dictated by the availability of job posting data, I use a **quarterly** time period. To aggregate, the **mean** value is used.

Let J_t denote the index set of job postings that fall within time period t . The aggregated similarity matrix $\bar{\mathbf{S}} \in \mathbb{R}^{n \times T}$ is

$$\bar{\mathbf{S}}_{r,t} = \frac{1}{|J_t|} \sum_{j \in J_t} \mathbf{S}_{r,j} \quad (5.7)$$

where T is the total number of time periods and $|J_t|$ is the number of postings in period t . Each entry of $\bar{\mathbf{S}}$ thus represents the mean similarity of repository r to all job postings in quarter t .

Chapter 6

Baseline (Keyword-based) Model

6.1 Motivation

In supervised learning, evaluation is straightforward because we have a **target** or ground truth: we know the correct outcome, so we can measure how well our model’s prediction matches that reality. Conversely, similarity analysis lacks such a priori definitions. There is no universal criterion by which two items may be deemed “similar,” and so evaluation requires task-specific labelling.

The utilized sentence-transformer models are trained on broad textual corpora [68], in order to capture and represent general-purpose semantics. Resultingly the semantic dimensions they capture may not align with our domain of interest. In particular, skill- and technology-specific relations are likely under-represented relative to more general linguistic patterns.

Since training a bespoke sentence-transformer model is not feasible given the available data, it is either necessary to fine-tune a pre-trained model, or to conduct an ablation-style study by modifying the inputs/outputs of pre-trained sentence-transformers, aiming to produce results that more closely align with our domain of interest. Since the former of these two options again requires high-quality & high-volume label data, I adopt the latter approach. Albeit evaluative baseline labels are still required, we can leverage a more lightweight labelling method.

Labelling here means constructing a set of examples (a job posting / repository pair) where the degree of similarity is known or agreed upon—either through expert annotation, crowd-sourced judgments, or some other reliable method. These examples serve as the **benchmark** or target, allowing for systematic comparison of different configurations to determine which one best captures the desired notion of similarity. It is therefore necessary to define labels before evaluating the efficacy of these representation methods.

6.2 Specification

In a general sense, the baseline model needs to be It is important to note that whilst we can make these comparisons across any pair of embeddings (e.g. $(\mathbf{r}, \mathbf{r})$ or $(\mathbf{j}, \mathbf{j})$), this paper concerns *cross-domain* similarity, so only $(\mathbf{r}, \mathbf{j})$ pairs are discussed.

I adopt the preference-ordering notation common in economics: $A \succ_q B$ indicates that, with respect to query q , A is preferred over B . Here, *preference* simply means higher similarity:

$$\mathbf{j}_a \succ_{\mathbf{r}_i} \mathbf{j}_b = s(\mathbf{j}_a, \mathbf{r}_i) > s(\mathbf{j}_b, \mathbf{r}_i)$$

Transitivity We will say that an important feature of what makes the similarity model *useful* for our case is to suggest that, for 3 job postings $\{\mathbf{j}_a, \mathbf{j}_b, \mathbf{j}_c\}$ and repository $\mathbf{r}_i$, the *transitivity* criterion maintains that:

$$\mathbf{j}_a \succ_{\mathbf{r}_i} \mathbf{j}_b \succ_{\mathbf{r}_i} \mathbf{j}_c \implies \mathbf{j}_a \succ_{\mathbf{r}_i} \mathbf{j}_c \quad (6.1)$$

Essentially, this claims that **ranking** is an important part of what makes our representation meaningful. By symmetry, the same transitivity should hold when ranking repositories with respect to a job posting:

$$\mathbf{r}_a \succ_{\mathbf{j}_t} \mathbf{r}_b \succ_{\mathbf{j}_t} \mathbf{r}_c \implies \mathbf{r}_a \succ_{\mathbf{j}_t} \mathbf{r}_c$$

This notion aligns with metric learning and triplet-loss frameworks [69], where a ranking is considered robust only if the similarity difference exceeds a margin δ :

$$\mathbf{j}_a \succ_{\mathbf{r}_i}^{\delta} \mathbf{j}_b \implies s(\mathbf{j}_a, \mathbf{r}_i) \geq s(\mathbf{j}_b, \mathbf{r}_i) + \delta$$

This prevents differences in similarities smaller than δ from being interpreted as meaningful, which improves generalization and reduces overfitting. However, I forgo this design-choice in light of the more compact range of similarity values obtained (see section 8 for these values).

The idea that transitivity must be maintained makes manually labelling pairwise similarities seem even more infeasible. Aside from the concerns of subjectivity, potential bias of human judgments and the time-intensive nature of evaluating all pairs, there would be great difficulty in maintaining a coherent, transitive ranking as new instances are considered or older judgments require revision. For these reasons we need an automated approach that preserves transitivity and is strongly grounded in its judgment of similar documents.

Truth above all In machine learning, predictions can be classified as True Positive (TP), True Negative (TN), False Positive (FP), or False Negative (FN); see the confusion matrix in figure 6.1.

		Prediction	
		<i>Positive</i>	<i>Negative</i>
Actual	<i>Positive</i>	TP	FP
	<i>Negative</i>	FN	TN

Figure 6.1: Confusion Matrix: A binary classification model is either correct (TP / TN) or incorrect (FP / FN). Optimizing the model means minimizing the *incorrect* judgements, but there exists a trade-off between FPs & FNs: Reducing one type of error often increases the other.

In an ideal model, FP and FN would both be zero. However, there is an inherent trade-off between minimizing these error types: reducing FP often increases FN, and vice versa [70].

For our baseline, it is critical that no FP occur — the model should never indicate high similarity between documents that are in fact unrelated. Minimizing FP requires adopting a conservative criterion for predicting similarity, which may result in some truly similar pairs being missed (FN). This trade-off is acceptable because the baseline is intended as a high-precision reference: any pair marked as similar is almost certainly so.

Operationally, the baseline uses a curated keyword dictionary, because of this high-precision requirement. Documents sharing at least one keyword in a given category are marked as similar (TP); documents sharing no keywords are considered dissimilar (TN). While some genuinely similar pairs may not share keywords (FN), the method prevents false positives (FP), providing a reliable foundation for evaluating the performance of more sophisticated embedding-based similarity measures.

6.3 Keyword Dictionary Design

To construct this baseline model, it was necessary to curate a list or **dictionary** of *domain-relevant keywords*. It is vital that this dictionary is constructed with

care, and is robust enough to capture important details.

The starting point for this task was to utilize the list of topic-tags present in the acquired GitHub repository data, which reduces subjectivity on my part to find those domain-relevant terms. Largely, these refer to domain/concept-level phrases (e.g. “machine learning”, “deep learning”), general tasks/roles (e.g. “Programming”, “MLOps”) or more fine-grained technologies and tasks (e.g. “PyTorch”, “Fine-Tuning”). A lot of these concepts are referred to by their acronyms, so the dictionary was constructed to equate both the verbose and acronym versions.

For a higher level of evaluative granularity, each dictionary entry is matched to a higher level category. This enables a downstream interpretation of the category-wise recall of semantic similarity representations.

Some tags are unrelated to the domain of interest - e.g. “audio” or “chinese”. The existence of these in the seed-list, which consists of all mentioned GitHub topics for the relevant repositories, is natural since they are not inherently domain-relevant in the sense of AI-related skills or technologies. By manual intervention then these unrelated tags are removed, acronym-term mappings are established, and each entry is mapped to one or more corresponding categories.

With the resulting dictionary of approximately 100 entries, an **LLM-assisted procedure** was employed for its expansion to ≈ 300 entries. The details of conducting this process are found in appendix A.1.3.2. While the GitHub seed-list provides a grounded starting point, it may omit synonyms, emerging terminology, and contextually relevant variants. LLMs, trained on large and diverse corpora, are capable of suggesting semantically related terms that complement the initial dictionary. In this sense, the model acts to add entries that share contextual usage with the seeds. Importantly, all expansions were subject to manual review to filter spurious or overly generic outputs.

6.4 Baseline intersection matrix

With the dictionary, we find the intersection of terms for pairs of documents. Formally,

$$\mathbf{I}_{i,t} = \{d \in D \mid d \in \mathbf{r}_i \wedge d \in \mathbf{j}_t\}$$

where D is the dictionary, and $\mathbf{I}$ represents the intersection matrix, whose entries are the sets of terms that a job posting and repository share.

It’s important to note here that d is *either* the dictionary entry’s acronym or verbose variant. This means that, for example, “AI” in one document is matched to “Artificial Intelligence” in another.

For the $d \in \mathbf{r}_i$ and $d \in \mathbf{j}_t$ matchings, an approach was leveraged whereby the text-content of job postings and repositories is digested in **lemmatized** form. This helps reduce, for example, plurals to singular (“AIs” to “AI”) and verbs to infinitives (“evaluating” to “evaluate”) to produce a more homogeneous textual space. Whilst this removes some syntactical understanding, semantic meaning is preserved under the notion that a term and its lemma are semantically equivalent.

Moreover, a **fuzzy** matching system was utilized to account for trivial mismatches (e.g. typos, hyphenations). Since the dictionary terms are relatively distinctive (e.g. ‘PyTorch’, ‘TensorFlow’), the likelihood of a spurious fuzzy match to an unrelated term is low. A high fuzzy-threshold was applied to further mitigate the potential for spurious matches, in line with the conservative philosophy of the baseline model’s construction.

Together these build a low-risk approach to relaxing the conservatism with which the keyword-matching finds shared terms. The full consideration of associated limitations are discussed further within section 10.4.

Chapter 7

Experimentation

Alternative configurations of sentence-transformer based representations were experimented with, to find the configuration that best captures the semantics of AI-skills & technologies. Herein lies a discussion of the experiments conducted.

7.1 Text Preprocessing

We know that the sentence transformer models capture general semantics, but we are only interested in specific semantic features. The texts of job postings and repositories do not purely consist of content relating to skills or technologies. In job postings, there are sentences that discuss the “Company Overview” or “Job Benefits”; in repository READMEs, there are sections on “License”, “Contributing” or “Installation”. We would regard these as *noisy* text with respect to the domain of interest, since it is likely that relevant features are obscured by such content at the embedding stage.

In testing the impact of *altering* input text, we use 2 variants in order to provide a controlled comparison between

1. raw-text embeddings (denoted **v1**), and
2. embeddings filtered by a bundle of intuitive heuristics (**v2**)

The purpose is not to optimize preprocessing in detail, but to test whether heuristic noise-reduction has *any* measurable impact on downstream similarity performance. More granular ablations would be informative, but are left for future work.

7.1.1 v1

Aside from removing punctuation and stripping whitespace, this text-variant is our “control” and resembles the raw text-content of documents.

7.1.2 v2

Shared heuristic Both repository and job posting text are stripped of “stop-words” (common & high-frequency words in a language, e.g. ‘and’, ‘is’, ‘the’) and are lemmatized.

Job postings heuristics Texts are stripped of *sentences* that do not contain proper-nouns (a good heuristic for capturing technology-related sentences) or terms that would generally appear in role-requirement sections (e.g. “skills” or “qualifications”).

GitHub repository heuristics GitHub repository README texts are **markdown** files, whose headings indicate sectional content - the irrelevant sections (e.g. “Installation” or “License”) are thus removed, as well as those that contain no semantic content (urls, hyperlinks or non-textual content).

7.2 Sentence Embedding Models

As mentioned, 3 variations of **pre-trained** sentence-models (tables 7.1 & 7.2).

Index	Model Name	Architecture	Dim.	Training Focus	Source
<i>a</i>	all-MiniLM-L6-v2	6 layers (~22M params)	384	Contrastive fine-tuning on large sentence-pair datasets	[68], adapted from [71]
<i>b</i>	all-mpnet-base-v2	MPNet: 12 layers (~110M params)	768	Contrastive fine-tuning; permuted language modelling	[72], adapted from [73][74]
<i>c</i>	bge-base-en-v1.5	BERT-style encoder: 12 layers (~109M params)	768	Contrastive learning with hard negatives; retrieval-oriented tuning	[75]

Table 7.1: Specifications of three sentence embedding models.

7.3 Dimensionality Reduction

High-dimensional sentence embeddings can encode generic features that obscure domain-relevant signals and increase computational cost. To address this, we apply Principal Component Analysis (PCA). The principal components of PCA are the eigenvectors of the embeddings’ covariance matrix, with the corresponding eigenvalues indicating the amount of variance captured along each direction. By

Model	Strengths	Trade-offs
all-MiniLM-L6-v2	• Very fast • Lightweight • Good general-purpose quality	• Lower accuracy than larger models • Less fine-grained semantics
all-mpnet-base-v2	• High accuracy on semantic similarity • Strong generalist	• Slower inference • Higher memory footprint
BAAI/bge-base-en-v1.5	• Excels at retrieval and search • Instruction-tuned for flexibility	• Less general-purpose than MPNet • Slower than MiniLM

Table 7.2: Strengths and trade-offs of three sentence embedding models.

projecting onto the top k eigenvectors, we retain the dimensions that explain the greatest variance while discarding lower-variance, potentially noisy directions.

Intuitively, PCA ‘rotates’ the high-dimensional embedding space to align the axes with directions of greatest variance, preserving the most informative patterns and reducing noise from less-variant dimensions.

We compute PCA on the vertical concatenation of repository and job-posting embeddings. Although the number of repositories is much smaller than job postings, both document types occupy the same semantic space, allowing principal components to capture variance meaningful across the corpus.

For experimentation, we produce embeddings with no PCA, $k = 64$, $k = 128$, and $k = 256$. These choices provide a meaningful range to assess whether dimensionality reduction affects the preservation of semantic information, with diminishing returns expected beyond 256 components.

Chapter 8

Evaluation

8.1 Introduction

In table 8.1 are the results of the experiments as evaluated against the baseline model. Recall that the baseline model dictates an intersection matrix $\mathbf{I}$, whose entries $\mathbf{I}_{i,t}$ are the set of custom dictionary terms shared between repository $\mathbf{r}_i$ and job posting $\mathbf{j}_t$. In addition, since each dictionary-term belongs to one or more **categories**, we can let $\mathcal{C}(d) \subseteq \{1, \dots, K\}$ denote the categories associated with term d . The binary matrix for category c can be defined as:

$$\mathbf{B}_{i,t}^{(c)} = \begin{cases} 1 & \text{if } \exists d \in \mathbf{I}_{i,t} \text{ such that } c \in \mathcal{C}(d), \\ 0 & \text{otherwise.} \end{cases} \quad (8.1)$$

For each job posting/repository pair (i, t) , we check whether **any shared term** belongs to category c . If at least one term in the intersection belongs to category c , we set the entry to 1; otherwise, 0. This naturally converts the intersection matrix $\mathbf{I}$ into a set of category-specific binary matrices $\mathbf{B}^{(c)}$.

Using 4 types of metrics, we can evaluate the performance of an **experiment run**, revolving centrally around their retrieval of proper **ranking**.

The terms ‘micro’ and ‘macro’ are used here to depict the element-wise preference of evaluation: the ‘macro’ computes first per-category then aggregates across job postings; the ‘micro’ reverses this ordering, aggregating across categories and computing per-job metrics. Both are viewed from the perspective as finally being aggregated across repositories. This is hopefully made more clear by demonstrating what each metric entails:

8.2 Evaluation metrics

Precision For a job posting j and category c , let repositories $r \in \{1, \dots, R\}$ be ranked by similarity scores $\mathbf{S}_{r,j}$ in descending order. With $\mathbf{B}_{r,j}^{(c)} \in \{0, 1\}$ as the binary indicator that repo r shares at least one term of category c with j , precision at rank i is

$$P_j(i) = \frac{1}{i} \sum_{k=1}^i \mathbf{B}_{r_k,j}^{(c)}. \quad (8.2)$$

and the **average precision** (AP) for j and c is

$$\text{AP}_j^{(c)} = \frac{1}{N_+^{(c)}} \sum_{i=1}^R P_j(i) \mathbf{B}_{r_i,j}^{(c)}, \quad (8.3)$$

where $N_+^{(c)} = \sum_{i=1}^R \mathbf{B}_{r_i,j}^{(c)}$ is the number of repos relevant to j under category c . Thus, AP measures how highly the model ranks repos that are truly relevant to j with respect to category c . The mean of this value across all job postings is, in table 8.1, called **Macro AP**.

A separate approach uses the sum of binary instances per (r, j) pair - i.e. as the *number of categories* that r shares with j (i.e. ‘Micro’ evaluation of AP). Given that

$$\mathbf{B}_{r,j}^{\text{micro}} = \sum_{c=1}^K \mathbf{B}_{r,j}^{(c)}$$

we’ll say that

$$P_j^{\text{micro}}(i) = \frac{1}{i} \sum_{k=1}^i \mathbf{B}_{r_k,j}^{\text{micro}} \quad (8.4)$$

In the table 8.1 I include the values of $P_j^{\text{micro}}(i)$ for $i \in \{1, 5, 10\}$, averaged across job postings (called **Micro P@1**, **Micro P@5** and **Micro P@10** in table 8.1). This gives a more granular view than AP, since we get a sense of how precise a run is at important ranks.

Again, we can compute the AP (all ranks):

$$\text{AP}_j^{\text{micro}} = \frac{1}{\sum_{k=1}^R \mathbf{B}_{r_k,j}^{\text{micro}}} \sum_{r=1}^R P_j^{\text{micro}}(i) \mathbf{B}_{r_k,j}^{\text{micro}}. \quad (8.5)$$

We call the mean of this value across all job postings **Micro AP (mean)**. It is useful to use both measures because they indicate differences in how well representation method recalls the proper ranking: If $\text{micro} > \text{macro}$, the run does well on dominant categories but struggles with long-tail ones where term matches are rare. If $\text{macro} \approx \text{micro}$, the run generalizes more evenly across categories.

This is indicative of a trend within table 8.1, since **Micro AP (mean)** is *always* higher than **Macro AP**, indicating that certain categories are matched more fre-

quently than others.

Mean Squared Error (MSE) weighted Let $\mathbf{B} \in \{0, 1\}^{R \times J \times K}$ be the binary tensor of relevance labels (stacking the binary matrices $\mathbf{B}_{(c)}$ for all categories c), and let $\text{prev}(k) = \sum_{r,j} \mathbf{B}_{r,j,k}$ denote the prevalence of category k .

We define weights

$$w_c = \frac{\frac{1}{\text{prev}(c)}}{\sum_{k \in K} \frac{1}{\text{prev}(k)}} \quad (8.6)$$

This needs adapting slightly to account for categories where $\text{prev}(k) = 0$. Let k belong to $\mathcal{K}$ if $\text{prev}(k) > 0$. Therefore, the weights are:

$$w_c = \begin{cases} \frac{\frac{1}{\text{prev}(c)}}{\sum_{k \in \mathcal{K}} \frac{1}{\text{prev}(k)}} & \text{if } c \in \mathcal{K}, \\ 0 & \text{otherwise,} \end{cases}$$

Rarer categories are hence given higher weight (absent categories are discounted to avoid distortions). The weighted binary labels for a job–repo pair are

$$\mathbf{B}_{r,j}^{\text{weighted}} = \sum_{k=1}^K w_k \mathbf{B}_{r,j,k} \quad (8.7)$$

$\mathbf{B}^{\text{weighted}}$ is therefore composed of the sum of matched categories, with rarer categories upweighted.

Let $\mathbf{S} \in \mathbb{R}^{R \times J}$ denote the similarity scores, rescaled into $[0, 1]$ (akin to using scikit-learn’s `StandardScaler`):

$$\mathbf{S}_{r,j}^{\text{scaled}} = \frac{\mathbf{S}_{r,j} - \min(\mathbf{S})}{\max(\mathbf{S}) - \min(\mathbf{S})}. \quad (8.8)$$

The weighted mean squared error is then

$$\text{MSE}_{\text{weighted}} = \frac{1}{RJ} \sum_{r=1}^R \sum_{j=1}^J \left(\mathbf{B}_{r,j}^{\text{weighted}} - \mathbf{S}_{r,j}^{\text{scaled}} \right)^2. \quad (8.9)$$

This metric (denoted **MSE Weighted vs. Similarity**) assesses whether similarity scores are numerically calibrated to the true overlaps between job postings and repositories. Intuitively, the MSE finds the mean of squared differences between weighted-category-matches and similarity scores. A lower value indicates lower differences overall, or just that high differences are less frequent. The $\mathbf{B}^{\text{weighted}}$ is higher when either more categories are matched or when rarer categories are matched, which in either case make a high similarity value desirable.

Normalized Discounted Cumulative Gain (NDCG) For each job j , define the *micro relevance* of a repo r as the sum of shared categories:

$$y_r = \sum_{c=1}^K \mathbf{B}_{r,j,c}$$

Let the repos be ranked by similarity $\mathbf{S}_{r,j}$, and let r_i denote the repo at rank i . The Discounted Cumulative Gain at rank k is

$$\text{DCG@k}_j = \sum_{i=1}^k \frac{2^{y_{r_i}} - 1}{\log_2(i + 1)}.$$

The Ideal DCG is computed by ordering repos by descending y_r (i.e. the ‘true’ score in our baseline model):

$$\text{IDCG@k}_j = \sum_{i=1}^k \frac{2^{y_{r_i}^*} - 1}{\log_2(i + 1)},$$

where $y_{r_i}^*$ is the i -th largest relevance (as per the baseline) for job j . Then, the normalized DCG is

$$\text{NDCG@k}_j = \frac{\text{DCG@k}_j}{\text{IDCG@k}_j}.$$

Finally, the mean (micro) NDCG@k is averaged across all jobs:

$$\text{NDCG@k}^{\text{mean}} = \frac{1}{J} \sum_{j=1}^J \text{NDCG@k}_j.$$

Effectively, the DCG evaluates the baseline model’s scores of each repository when ranked by the *similarity* model, and the IDCG uses the *actual* of the baseline model. We still place value on the correctness of lower ranks, but these logarithmically discounted in favour of higher ranks.

This is akin to querying $\mathbf{S}$ per job posting (column), to predict the top- k most similar repositories, scored on how well it matches the actual ranking from the baseline model, and favouring correctness for the higher ranks. We can measure nicely both how well an experiment run performs with top k ranks, and especially those higher ranks (the *most* similar repositories).

This, combined with the same k value of $P_j^{\text{micro}}(k)$ (averaged over job postings), gives an indication of how weighted ranking compares to non-weighted ranking. For example, if $\text{P@k} < \text{NDCG@k}$, then the correct predictions within k are distributed towards the top.

I use **Micro NDCG10 (mean)** with $k = 10$ in the evaluation, though other k are viable choices too - for simplicity I forgo analysis of other values, albeit to some

extent neglecting a more wholistic outlook.

8.3 Evaluation results

Depicted here in table 8.1 are the metric values of evaluating the various representation experiments on the baseline model.

Table 8.1: Evaluation of experiments: the various configurations (runs) of the sentence-embedding inputs & outputs are evaluated on various metrics. The v2_c run appears strongest.

Run ID	Macro AP	MSE Weighted vs. Similarity	Micro AP (mean)	Micro NDCG10 (mean)	Micro P@1	Micro P@5	Micro P@10
v1_a	0.285	0.157	0.804	0.798	0.819	0.797	0.790
v1_b	0.291	0.160	0.824	0.843	0.886	0.846	0.823
v1_b_pca256	0.305	0.143	0.814	0.824	0.859	0.823	0.811
v1_b_pca64	0.306	0.182	0.814	0.824	0.855	0.823	0.811
v1_b_pca128	0.304	0.160	0.814	0.824	0.854	0.824	0.811
v1_c	0.292	0.140	0.826	0.847	0.869	0.844	0.831
v2_a	0.291	0.231	0.833	0.852	0.876	0.854	0.838
v2_b	0.284	0.182	0.829	0.855	0.851	0.855	0.843
v2_b_pca256	0.299	0.158	0.818	0.835	0.857	0.834	0.822
v2_b_pca64	0.299	0.176	0.818	0.835	0.859	0.836	0.822
v2_b_pca128	0.298	0.153	0.818	0.835	0.860	0.835	0.822
v2_c	0.304	0.201	0.850	0.877	0.929	0.875	0.852
v2_c_pca128	0.301	0.179	0.819	0.836	0.859	0.839	0.823
v2_c_pca256	0.301	0.166	0.818	0.835	0.857	0.838	0.822
v2_c_pca64	0.302	0.189	0.820	0.836	0.861	0.839	0.823

8.4 Discussion

As can be seen, the representation method that broadly performs best is v2_c: Though v1_b_pca64 and v1_b_pca256 are marginal improvements in Macro AP (0.306 and 0.305 compared to v2_c’s 0.304), run v2_c outperforms in all ‘micro’ domains.

This sentence model, BAAI/bge-base-en-v1.5 is a base-scale sentence-transformer model that is architecturally rooted in encoder-only (BERT-based) design. As a v1.5 release (September 2023), it delivers more balanced similarity distribution and enhanced retrieval capability, addressing prior shortcomings in responsiveness and semantic alignment. Its modest size means it is still computationally efficient,

whilst maintaining state-of-the-art performance on benchmarks like MTEB (Massive Text Embedding Benchmark) [76], which evaluates text embedding models on 56 datasets spanning 7 task categories. Unlike general-purpose transformer models such as `all-mpnet-base-v2` and `all-MiniLM-L6-v2`, it is specifically trained for text embedding and semantic similarity tasks, making it well-suited for information retrieval, clustering, and **cross-domain representation**. So it is unsurprising that this is superior to those results from general-purpose models.

As mentioned in section 6.2, we opt to *not* use the δ method for more strict similarities since there seems to be marginal of difference between many of methods, so more sharp distinctions between similarity values is necessary. Its only caveat is having a seemingly much higher MSE (0.201) than the best in this respect (its `v1` sibling at, 0.14). In fact, it can be seen that the MSE values are generally higher for `v2` than for `v1`, suggesting that perhaps the text preprocessing method for noise-reduction is slightly exaggerated, neglecting important concepts that would align with dictionary terms. Another interesting result here is that there appears to be no tangible benefit to using PCA - for example, `v2_c` variants experience drops in precision performance across the board when we reduce dimensionality. It may yet be a worthwhile design-choice with if paid greater attention in experimentation, but the results nonetheless depict that similarity representations in their full-dimensional form best reflect the dictionary-term rankings.

Chapter 9

Cross-domain Correlation Analysis

9.1 Introduction

The final stage of the methodology for this paper concerns the analysis of derived similarity time-series against that of the repositories' popularities. Recall that from equation 5.7, $\bar{\mathbf{S}} \in \mathbb{R}^{R \times T_j}$ for R repositories and T_j time periods - this adapted notation reflects the range of time-periods $t \in \{1, \dots, T_j\}$ that our job postings data exhibits.

Similarly, we can construct the matrix $\mathbf{P} \in \mathbb{R}^{R \times T_{\text{star}}}$ to represent the aggregation of stars (number of new stars) per equivalent time-period for each repository $r \in \{1, \dots, R\}$. T_{star} differs from T_j in that many of the examined repositories are older than the earliest job posting.

Nevertheless, we can align these matrices and evaluate their trends accordingly.

9.2 Analysis

Given the surface level nature of this analysis task, we choose to assess some basic correlation features between rows of $\bar{\mathbf{S}}$ and $\bar{\mathbf{P}}$. We are examining, for a given repository, to what extent its popularity trend is shared with the trend in similarity to job postings.

It is noted that this analysis is not thorough, so I refrain from drawing any major conclusions on the basis that there's much more to assess. Given that, the scope of this paper is primarily focused on building strong representations, this analysis stage of the pipeline is afforded less importance. It is left to future work to investigate more rigorous the statistical significance of *causality* or *lead-lag* effects.

Pearson & Spearman Correlation The *Pearson correlation coefficient* measures the strength of a *linear* relationship between two variables, while the *Spearman correlation coefficient* measures the strength of a *monotonic* relationship (based on ranks).

For example, consider

$$y = 2x \quad \text{and} \quad y = 2^x.$$

The linear function yields $\rho_{\text{Pearson}}(x, 2x) = 1$, since the relationship is perfectly linear. For the exponential function, the relationship is strictly monotonic increasing, so $\rho_{\text{Spearman}}(x, 2^x) = 1$, but the Pearson correlation $\rho_{\text{Pearson}}(x, 2^x) < 1$, since the mapping is nonlinear.

Lags Considering that the broader idea of this study concerns innovation diffusion over time, we would not expect to see that the time-series for popularity & similarity move together in time. Rather, we would expect that trends in popularity *precede* those in similarity.

To this effect, we would conduct a **lagged** time-series analysis by shifting similarity scores backwards in time. Formally,

$$\bar{\mathbf{S}}_{r,t}^{(k)} = \bar{\mathbf{S}}_{r,t-k}$$

for a lag k .

Visually, we can imagine that shifting $\bar{\mathbf{S}}$ by k and examining its resulting correlation with $\mathbf{P}$ is like sliding the pattern of $\bar{\mathbf{S}}$ backwards over that of popularity, to find the shift which aligns those patterns best. Figure 9.1 illustrates this nicely with the example of correlations between lagging cos and sin.

Two variables that are then interpreted here are *Best Lag*, corresponding to the time-shift that produces the *Max Correlation* of the similarity & popularity time-series.

9.3 Discussion

The figures for this section suggest some interesting notions that would seem to confirm prior expectations.

9.3.1 Mean values

To start, figure 9.2 shows the average values of popularity & similarity per repository.

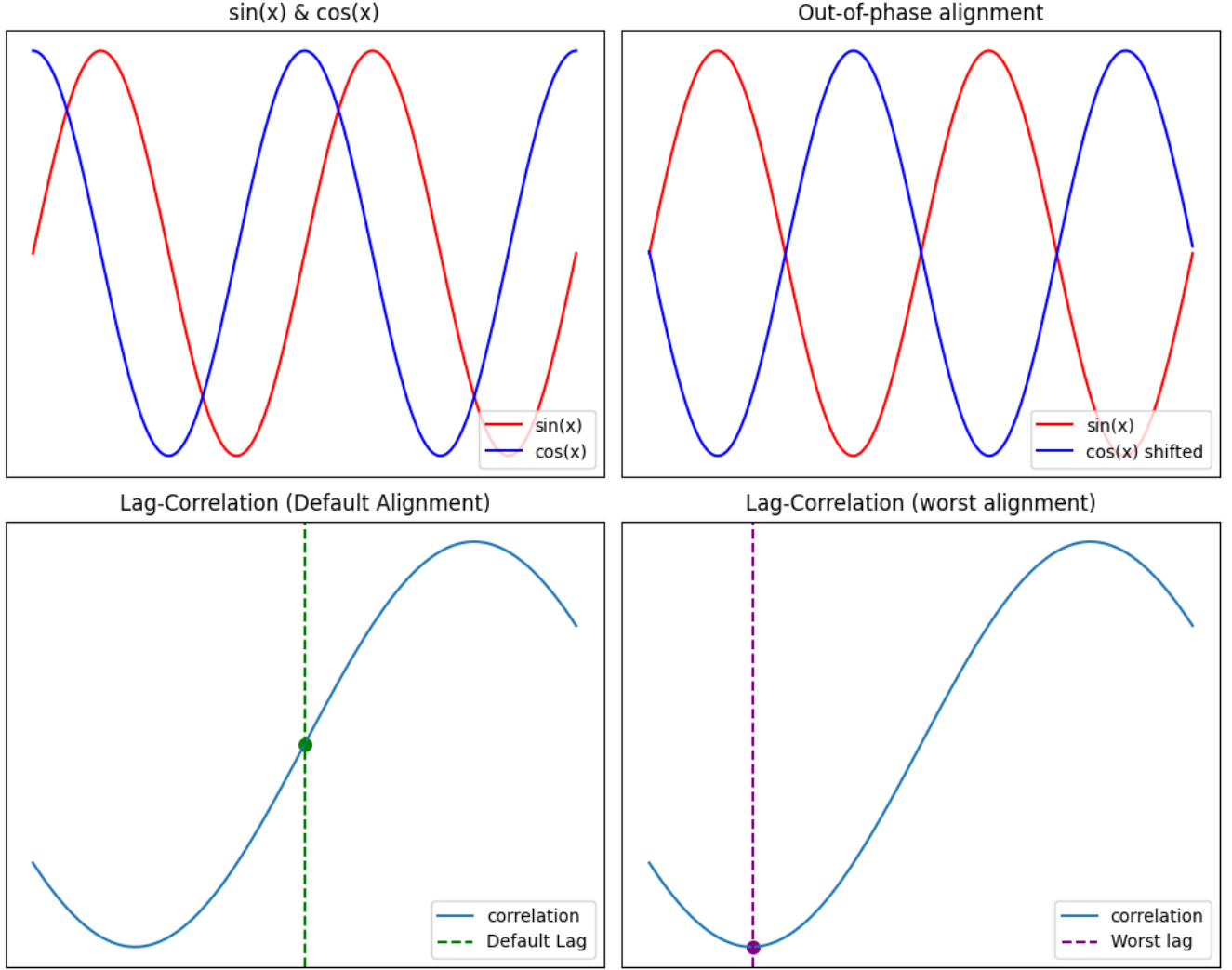


Figure 9.1: Lag Correlation example: $\cos$ & $\sin$ are ideal examples to demonstrate the notion of lagging one sequence to find different lag-correlation values with another

This would seem to suggest that except for a few outliers, the mean popularity is fairly low - recall that “popularity” here is the *stars gained during a quarterly period*, and the mean is relevant to repository age. That is, the mean popularity is

$$\bar{P}_r = \frac{1}{|T_{\text{Age}_r}|} \sum_{t \in T_{\text{Age}_r}} P_t$$

where T_{Age_r} is the time at which that repository was created.

We can also see that mean similarity takes a much broader range, indicating that some repositories are not deemed qualified as similarly as others (Mean similarity is calculated similarly, only for those periods where there is non-zero similarity; i.e. the periods with *any* job posting data). There are a number of reasons why this might be:

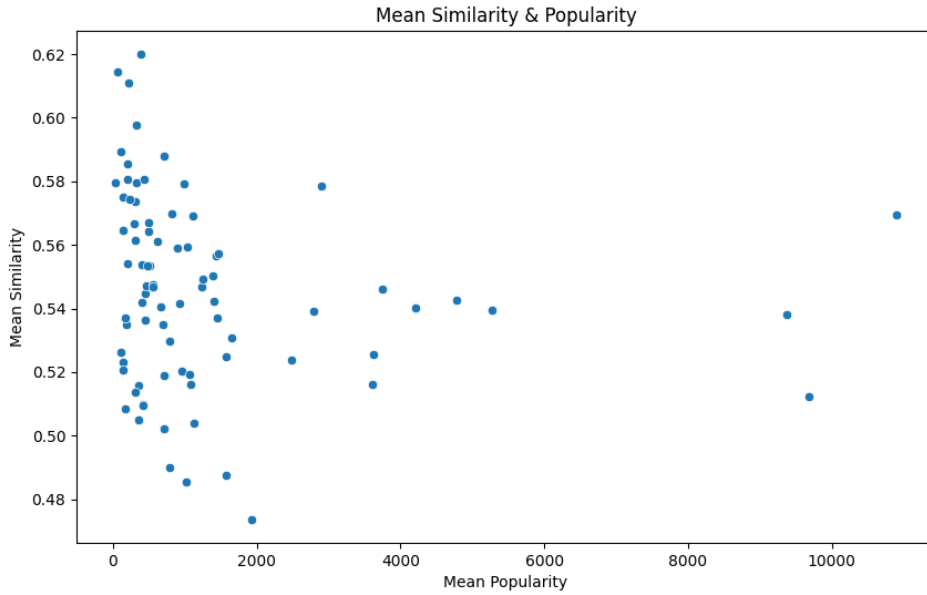


Figure 9.2: Scatterplot of mean-popularity vs. mean-similarity: most repositories have low mean-popularity, with a fairly broad mean-similarity

- Some repositories are indeed less relevant because of their niche nature
- Some repositories contain many more patterns relevant to the baseline-dictionary, thereafter with the sentence-embedding representation

The extent to which we can distinguish between these is limited due to the nature of how our similarity representation was constructed. Limitations to interpretation such as this are further discussed in section 10.4. Note that also, we see no negative values for similarity; this is suggestive of something deeper within the semantic embeddings that is not sufficiently expressive of *non-related* texts in our setting, and suggests that generic linguistic patterns still dominate the representations to some degree.

Nevertheless, together with the box-plots in figures 9.3 & 9.4, we can appreciate that there is avid variation within and between repository similarities (as they compare to job postings).

Some of the box-plot results appear surprising - ‘Netflix/Metaflow’ has the highest mean similarity, for example (figure 9.4). To some extent this idea of ‘mean similarity’ is not particularly relevant: under our fundamental criteria, what matters is that the *ranking* is coherent and consistent - equation 6.1 suggests that what matters is the *relative* relevance of a job posting to a repository, as opposed to some “absolute” value of relevance. Equivalently the same comparison is

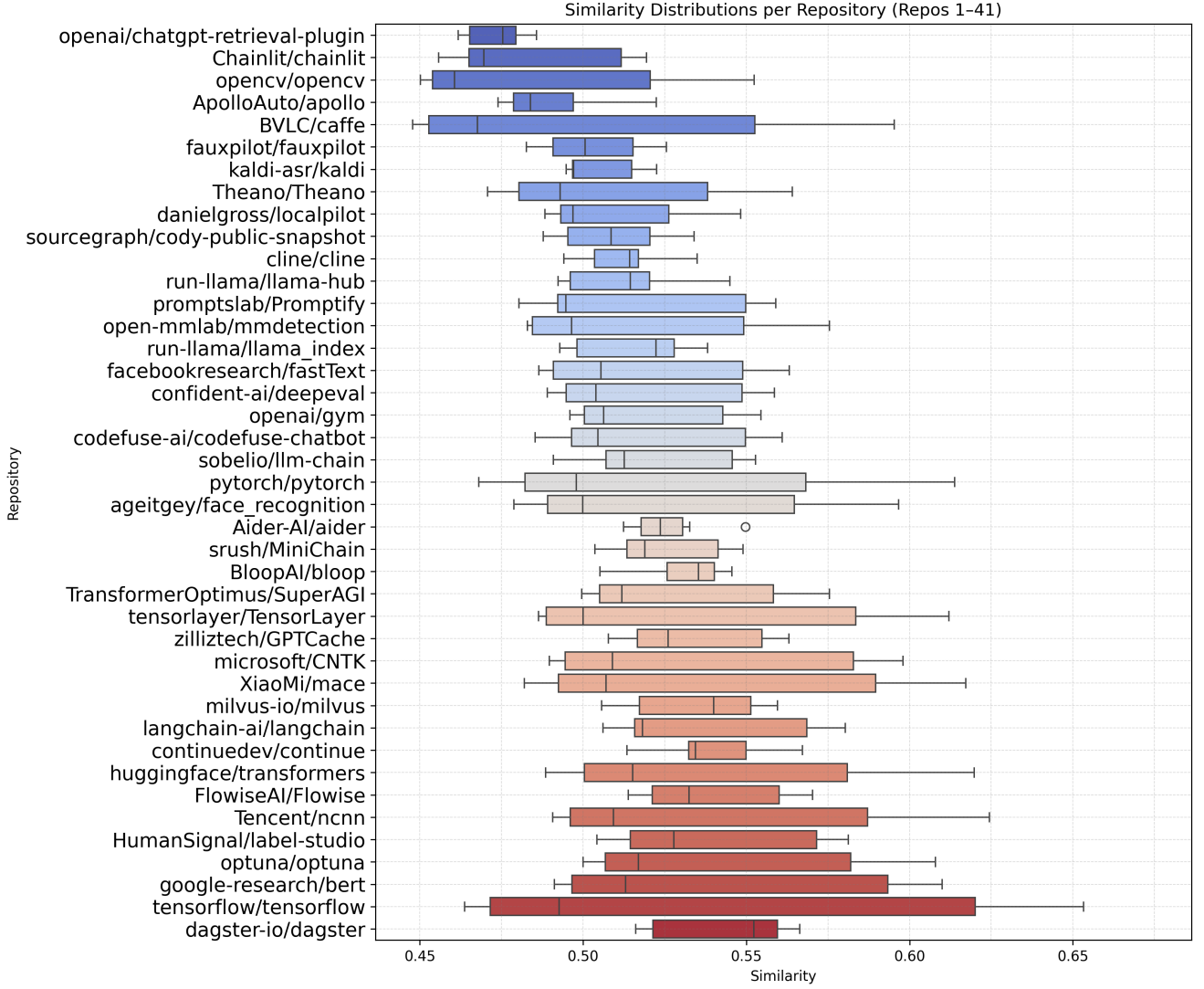


Figure 9.3: (Part 1) Boxplot of repository similarity values, ordered by mean similarity: This shows the range of similarity values that each repository expresses.

important in the other direction (repository vs. repository with respect to a job):

$$\mathbf{r}_a \succ_{\mathbf{j}_i} \mathbf{r}_b$$

9.3.2 Correlations

Without any lags, the correlation of similarity versus popularity is not indicative of any deeper trend (figure 9.5).

We can see that there are roughly as many repositories with high correlations as there are low correlations, which is not indicative of trends at time-aligned series.

However, when we compute the correlations exhibited at lagged periods, we can see that this distribution shifts dramatically (figure 9.7). The best way to interpret this is that when we allow for the similarity trends to align with older

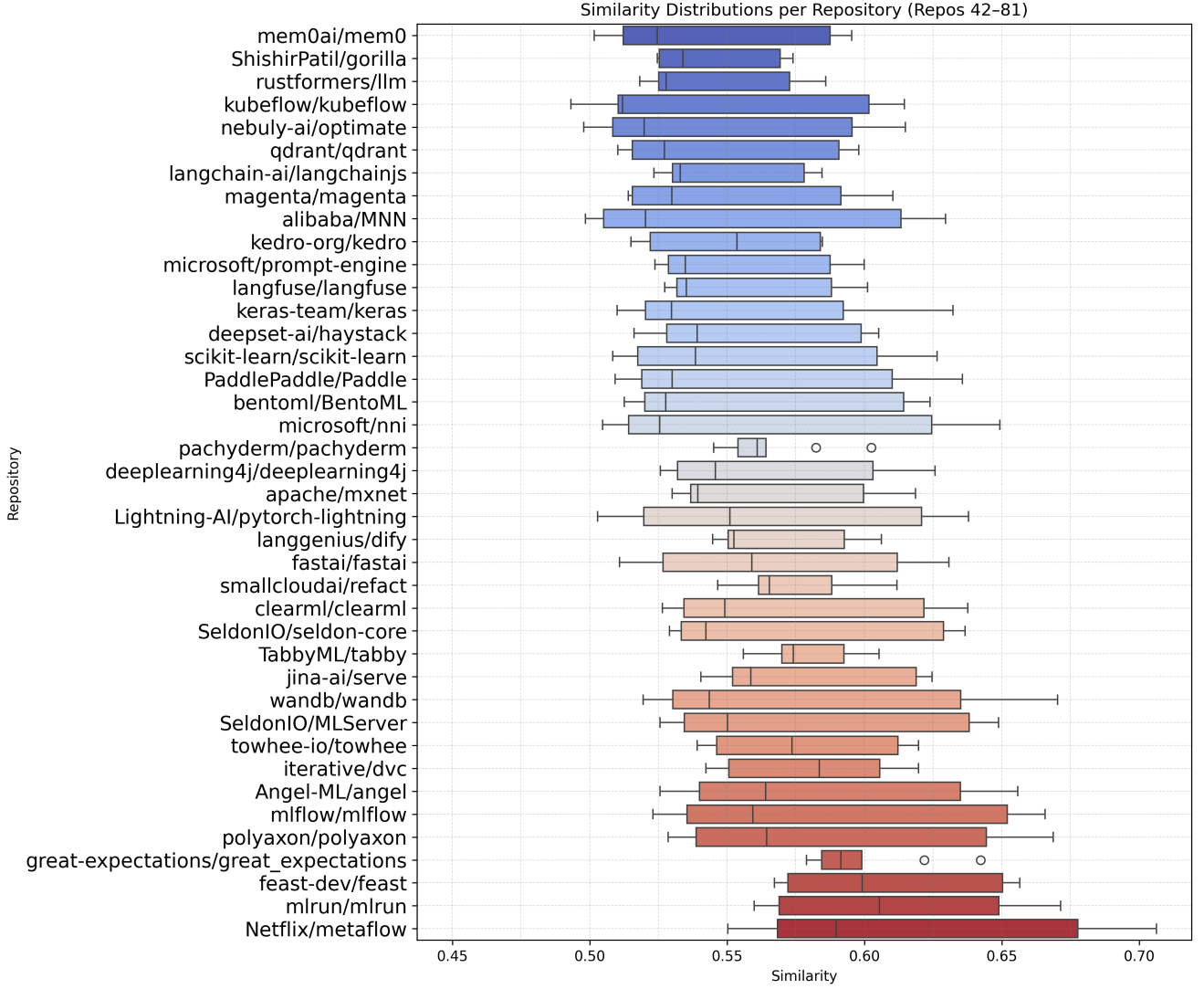


Figure 9.4: (Part 2) Boxplot of repository similarity values, ordered by mean similarity: This shows the range of similarity values that each repository expresses.

time periods, most repositories demonstrate a higher correlation between similarity & popularity.

For example, if a repository r has its ‘best’ lag at $k = 4$, then we would say that “The trend in similarity between r and job postings *today* is correlated best with its trend in popularity *a year (4 quarters) ago*”.

We can investigate further the nature of these ‘best’ lags (figure 9.7). As is fairly evident, repository age naturally constrains the maximum lag, since beyond this there is no repository to speak of. What is interesting, however, is that for older repositories, the best correlation to similarities exists at large lag periods (i.e. with trends in popularity from much earlier in their development).

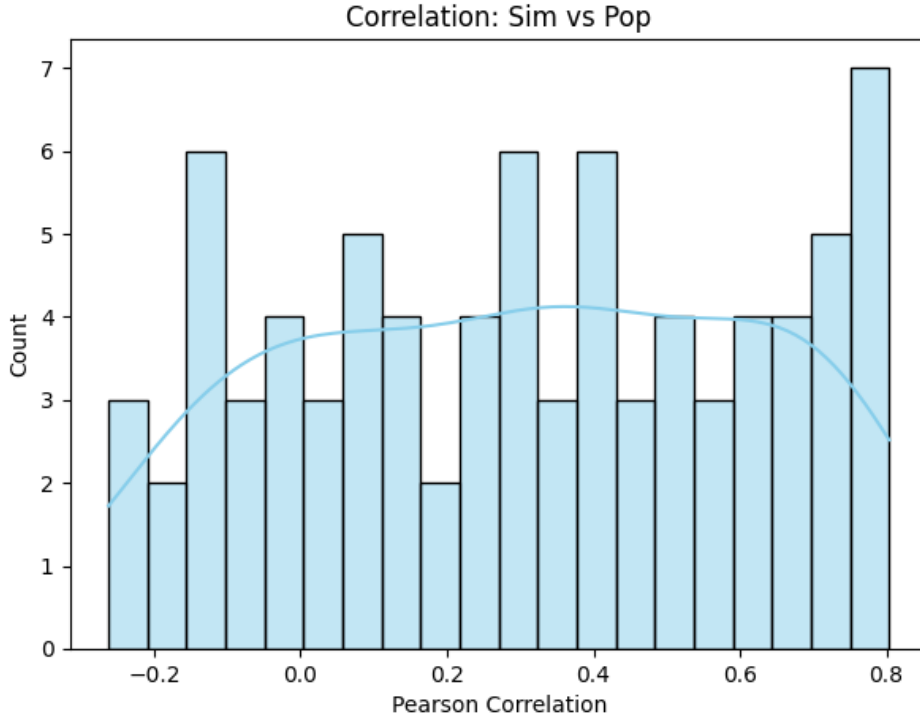


Figure 9.5: Distribution of correlation values (Similarity vs. Popularity): An even distribution of correlations, more random than meaningful

9.3.3 Broader Trends

There’s a distinct narrative that appears to surface with this, and other, insights: older repositories are less indicative of trends that occur in more recent periods. One explanation for this is that at their time of creation, AI technology was in relative infancy, and their proliferation in job postings only follows to a large extent after an explosion in innovation (as is demonstrated by the large number of young repositories). Figure 9.9 is a high-level view of a broader trend in this sense - we see that though there are some older repositories (Figure 9.8 demonstrates the distribution of repository ages), the majority are somewhere between 6 and 9 years old (24 – 36 quarters)¹, with a burst of new repositories in and around 2 to 3 years ago (8 – 12 quarters). It is seen that broadly, repositories peak in popularity (periods with highest rate of stars gained) soon thereafter their creation, which confirms that found by the authors of [36], [37]. Importantly, there is a big leap in new repositories and popularity in the range of 2 to 3 years previously (roughly, in April of 2023).

This jump is seemingly most explanatory of the leap seen in similarities, relating to the prevalence of the technologies in job postings. A narrative here might

¹Note that I discuss the *age* of a repository, but the *x*-axis of figure 9.9 indicates *time* of creation. The age is simply n_{periods} (61) minus the creation-time.

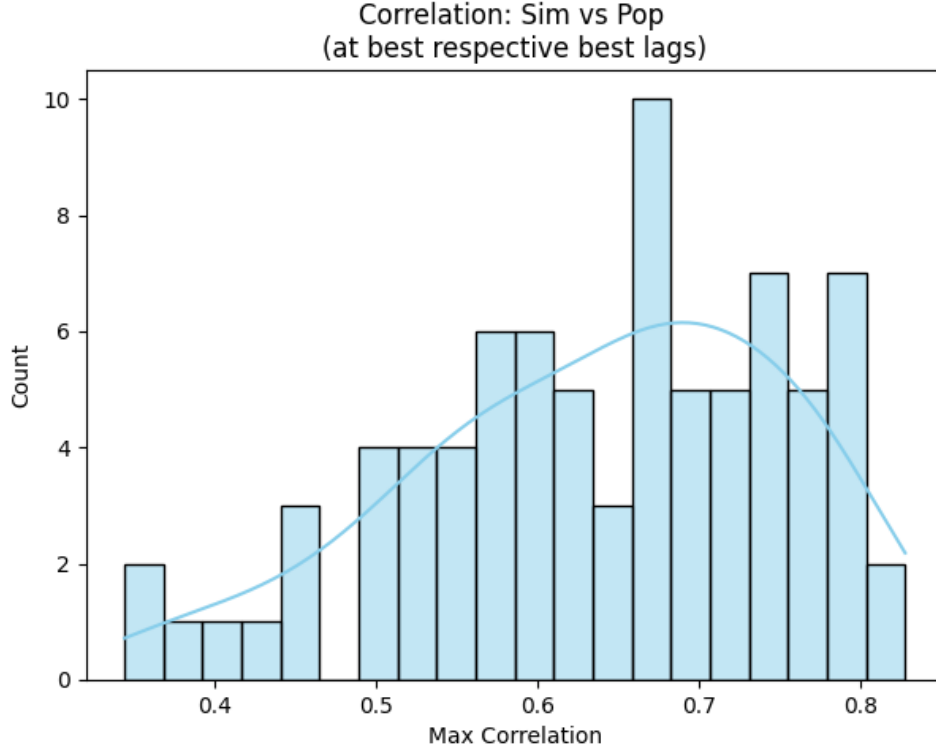


Figure 9.6: Distribution of correlation values (Similarity vs. Popularity) at respective *best lags*: Most repositories have a high-level of correlation when factoring in time-lags

suggest that some underlying popularity trend infused many ideas & innovations in the repository space, which thereafter is adopted by demands for related skills in job postings. Future work would consider expanding the pipeline to include *other* data sources, potentially aiding the explanation of this explosion in repository creations at this time. More concrete & rigorous analysis of such speculations is necessary, but is left for future work given the exploratory scope of this analysis.

9.4 Summary

To summarize then, the method depicted in figure 3.1 is carried out and we find that:

- A baseline model is developed using a balance of conservative & broad-capture heuristics that derive from a keyword-matching dictionary, containing domain-relevant terms.
- Using `bge-base-en-v1.5` and the `v2` text preprocessing method returns the best representation of (repository/job posting) sentence-transformer embedding similarity among alternatives, as depicted by its evaluation against the

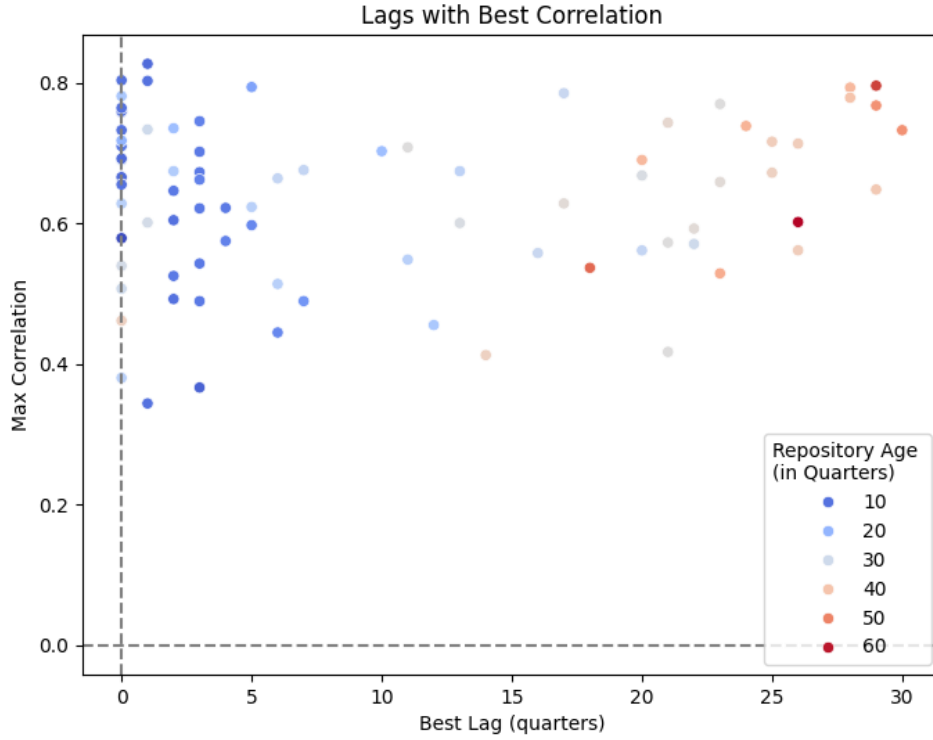


Figure 9.7: Distribution of correlation values (Similarity vs. Popularity) at respective *best lags*, coloured by repository age

baseline model. PCA does not appear to improve results, contrary to the belief that generic-features are noisy.

- The similarity & popularity matrices represent per-repo time-series, analogous to innovation-diffusion representations.
- Analysing these time-series gives insight into the nature of the AI-technology development & demand, most distinctly highlighting the burst of adoption that follows from the explosion in innovation some periods prior.

It is important to acknowledge that due to the plurality of analytical tasks, each specific aspect deserves greater attention to more rigorously understand their dynamics. Our objectives were to build a broad ‘proof of concept’ pipeline that builds a platform for meaningfully connecting the information within unstructured text data, relating OSS innovation to professional adoption in a novel fashion.

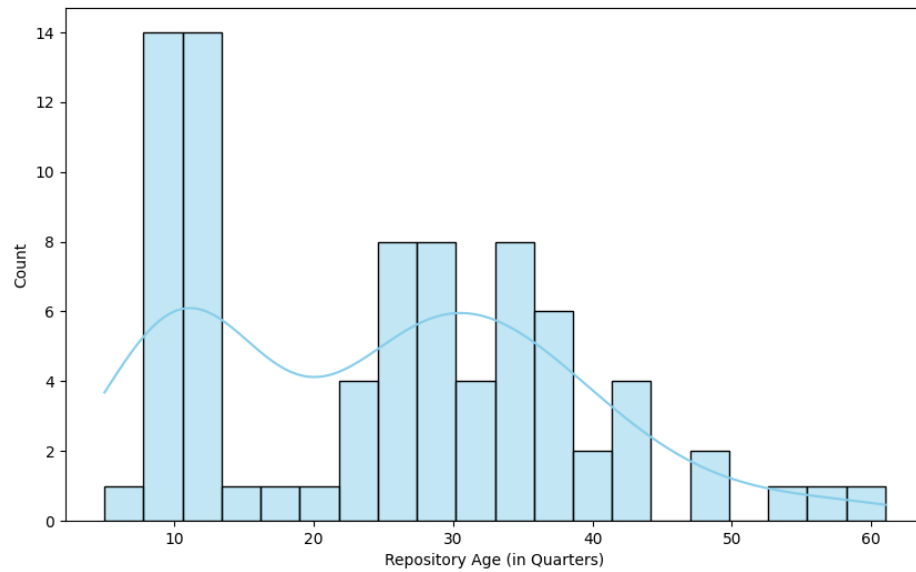


Figure 9.8: Age of Repositories: The distribution demonstrates that a burst of related repositories are created $\approx 10 - 12$ quarters ago (between 2 and 3 years - roughly in the 1st and 2nd quarters of 2023)

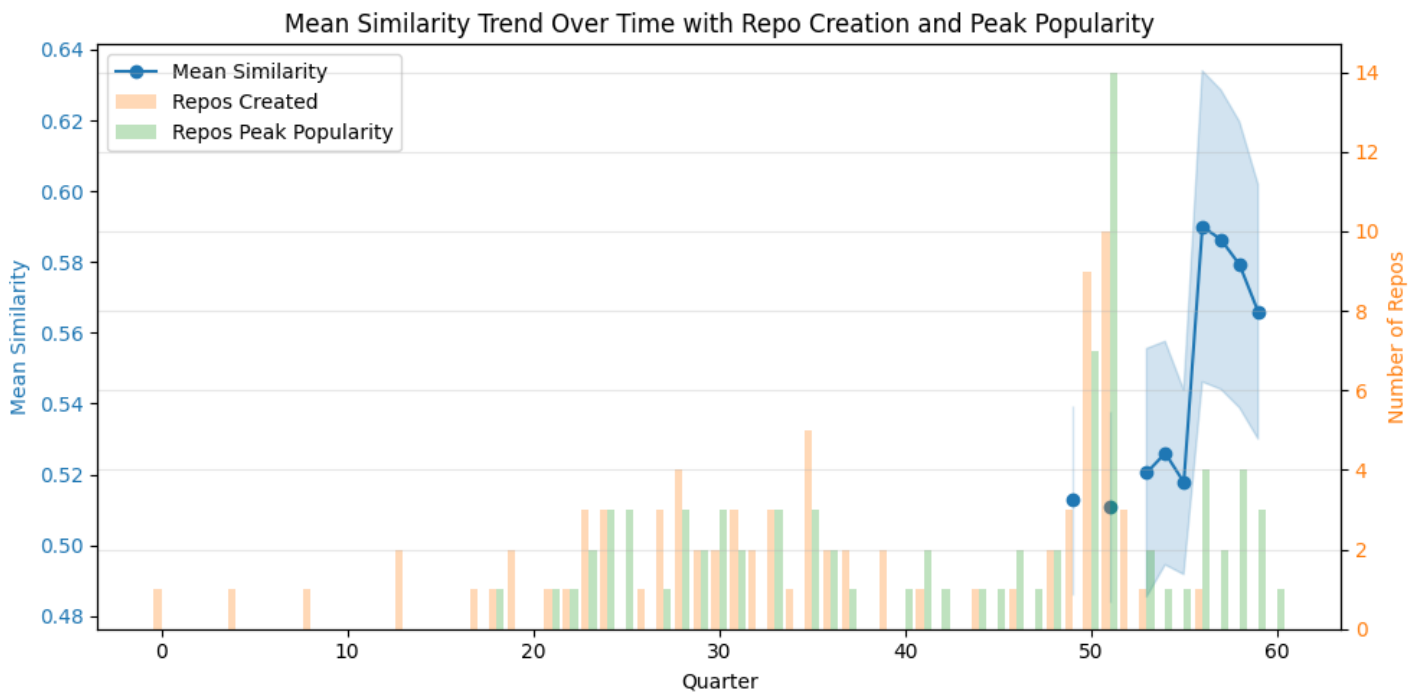


Figure 9.9: Aggregate trends: Orange bars indicate number of repositories created at that time period; Green bars indicate the number of repositories that peak in quarterly popularity; blue lines indicate the mean similarity across all repositories, with the standard deviation

Chapter 10

Critical Evaluation

10.1 Project-Level Problems

Here I'd like to discuss, in brief, some higher level challenges faced over the course of this research project.

10.1.1 Research Goal & Directions

In initial stages, the idea for the project concerned itself more so with the time-series element, and investigations/analyses therein entailed. It was clear that this would require deriving a highly robust time-series representation of GitHub repositories and job postings as a function of their shared skills, but only as time went on did it become clear how challenging this itself would be.

As a result, the focus of the project was modified, in order to account for the need to develop a high quality representation component. This whole reorganization unfortunately reflects lost time & effort, which in hindsight is easily dismissed as poor forethought. In future, it would perhaps be more effective to build out from a smaller research question (How do we represent a GitHub repository and a job posting as a function of their shared skills?), as opposed to reaching backwards from a theorized end-goal (forecasting skill demand).

10.1.2 Code organization

Attributable in part to the change in research direction, managing the codebase for this project proved difficult. My initial instinct was to develop code experimentally, exploring options for implementation along the way. As such, there was no overarching structure that would inform later developments. As the codebase grew, it became necessary to restructure and define a framework *a posteriori*, since the goal of this project was to develop a 'pipeline' that is readable, maintainable

and robust.

I’ve learned then that keeping meaningful documentation practices is vital for managing large codebases, and that in future work, an important first task is clearly defining a project framework before writing *any* code.

10.2 Strengths

Though imperfect, the primary strength of this study is its novelty. There is a serious lack of literature concerning this problem domain (namely that of linking software innovation to skill demand), and I believe that this paper shows that there is value in its investigation.

Moreover the ‘pipeline’ depicted follows a structured and thoroughly reasoned approach, guided by well-proven techniques. This notion of a pipeline is valuable in of itself because it represents an end-to-end solution, each component of which can be modified or expanded upon, thereby delivering a framework for data-driven analysis that is useful for many audiences (e.g. employers, policy-makers or job-seekers).

10.3 Assumptions

A facet of the limitations that such studies is the assumptions that they embody. Here I’ll discuss some of the problematic assumptions that I have identified, with a justification for their presence.

10.3.1 GitHub repositories as proxies for OSS

I’ve framed this study as one which aims to examine the relationship of software **innovation** and **adoption**, particularly as it concerns AI. The former of these ‘variables’, as it were, places a focus on open-source software. This as it stands is reasonable, in that OSS ecosystems are widely held to represent catalyst beds or ecosystems of innovation in software. The leap that *I* make thereafter is to treat **GitHub repositories** as representing OSS in totality, disregarding those other platforms or sources of OSS that might equivalently be treated as important.

As already mentioned, this assumption is somewhat defensible because GitHub is by far the most dominant OSS platform. However, it may be disputed that the content examined is not that concerning software innovation, but instead the dynamics relating only to GitHub repositories. Of this paper would be in terms of the link between GitHub repositories and job postings, rather than OSS as a whole. However, I affirm the assumptions’ defensibility, thereafter retaining the

original framing of importance as a solution to a more general research question (concerning software innovation & adoption)

10.3.2 Job postings as proxies for labour demand

In a similar vein, the latter of the two ‘variables’ under examination (software *adoption*) is viewed from the perspective of an economic implementation. This is a major facet of what the diffusion and adoption of innovations entail. One method of observing economic implementation in this sense is to assess the uptake of required skills, or demand of such knowledge, in the job market; Another wave would have been to examine the nature of patent filings over time, relating to proprietary product development.

Either of these do indicate innovation-adoption signals on behalf of economic actors (such as firms or individuals), so with pursuing the idea of the skill demand, I choose to use *job postings* as a proxy for this measurement. The assumption that these are valid proxies is disputable on the grounds that job postings are only loosely, not intrinsically, indicative of underlying skill demand. What I mean by this is that firm advertising a job is not necessarily equivalent to a firm hiring for that job. Moreover, there is a question of duplication: A firm hiring for one vacancy may post identical advertisements, either over time or over separate job boards. As well as this, online advertisement for vacancies is not the only method of such advertisement. Many vacancies are advertised in offline formats, and recruitment/head-hunting is another (albeit more elusive) form of advertisement. Particularly in the latter point, we may face an issue of selection bias, since the most highly sought after skills I’m most frequently associated with high-knowledge individuals, the majority of whom are not job-seeking on job-boards but are instead approached directly as high-value talent by top firms.

For the most part, literature supports the idea that job postings are a valid proxy for labour demand, so this assumption is maintained accordingly.

10.3.3 Job Posting Source concatenation

A part of the representation pipeline concatenates all job posts into a single data set. Doing so involves homogenizing some characteristics that distinguish the separate initial data sets.

Examples of these characteristics include:

1. The original source: which site or job-board the data-points originate from (e.g. LinkedIn or Indeed)

2. Extra fields: Including items such as separate company and industry information.
3. Metadata characteristics: These would include facets such as the location of the job/posting.

As an example, 2 of the collated datasets, [63], [64], which were curated by the same institution, with the former concerning only vacancy advertisements in *Ireland* and the latter in the *USA*. It is likely that the rate of innovation adoption is region-specific, so this assumption is weak in the face of such realities. Moreover, the language of every job posting we use is English, which neglects a significant portion of the global job market.

Given that the judged impact of such issues is minimal, we therefore move forward by accepting each individual dataset as different time-slices of the same data population for the sake of simplicity. Broadly, this is because the impact or usage of OSS (from GitHub) is not region-specific, and as a result, so is the adoption of associated innovative software. The fact that **Python** was developed in the Netherlands did not hinder its proliferation in the global Tech community.

10.3.4 Job Title Relevance

With the associated data set of job postings, A filter was applied to extract only those with titles relevant to ML/AI. This, in hindsight, was a subjective and perhaps erroneous decision because it presupposes extracting relevance, which is precisely the purpose of the downstream representation method. I.e. a non-ML related job (e.g. “Junior Nurse”) should be dissimilar to repositories anyway, so it matters not whether they exist as part of the input data.

The reasoning for this followed from initial ideas of what the project would entail, concerning more a forecasting/time-series centred focus. In this sense, it would be prudent to only include jobs of relevant occupations, allowing for fair temporal comparisons of trends within those occupations.

Given that training a bespoke sentence-model (more expressive of skill- or technology-based semantics) lay beyond the scope of this project, This filtering of job titles was maintained under the premise that we are evaluating representation methods of their *intra*-industry specific skills. I.e. since we are interested in examining the relationship of certain skills/tech *within* AI-related software, it is justified that we extend the same criteria for which occupations are considered.

10.3.5 Using *OSS Insight* Collections

As discussed in the relevant methodology section, we use OSS insight as an information source. They curate ‘collections’ of trending repositories, thereby aiding in dictating which repositories would be useful for this AI/ML related analysis. It is acknowledged that this method is not perfect, since we are effectively outsourcing a subjective judgement to another party. To some extent, we can expect their judgement to be superior due to their community-based curation procedure, so we accept this as at least a strong heuristic for repository relevance.

Moreover, for downstream time series analysis, the inclusion of only ‘trending’ repositories can be seen as *selection bias*: We can’t draw conclusions on broader trends of GitHub repository popularity on job postings if we only consider those repositories that are *a priori* judged to be popular.

The reason that I dismiss such criticisms here is because I refrain from making broader claims - the results are discussed purely in terms of the repositories that are present. Moreover, it is a pragmatic choice to utilize only a subset of relevant repositories, due to the excessive computational resources presented by inputting larger sets of repositories.

10.4 Limitations

Here I discuss the limitations faced by the implementation aspects of this project.

10.4.1 Data Limitations

As previously discussed, a fundamental limitation to the success of this paper’s method concerns the availability of job posting data. Not only is this a hinderance in the sense that we get incomplete snapshots of any time period, but the existence of data at many time periods is sparse or non-existent.

Figures 4.1 & 4.2 describe the problems posed here. Effectively, this limits the extent to which downstream correlation analysis tasks can be truly informative - the age of many repositories far exceeds that which we have in available job posting data, making those analyses lack sufficient detail.

10.4.2 Relying on General-Purpose Sentence Transformers

As alluded to, the principal trade-off in relying on pre-trained sentence models is that they may not capture the domain-specific semantic nuances of my application

with complete fidelity. These models are trained on broad, heterogeneous corpora and thus excel at general-purpose semantic similarity.

However, they may under-represent subtle distinctions that are critical within specialized domains (e.g., technical terminology, fine-grained skill taxonomies). In such cases, domain-adaptive pre-training or training a bespoke model from scratch could, in principle, yield embeddings more finely attuned to the task.

Such an undertaking requires a large amount of high-quality data, with proper target-labelling, that is time-intensive and expensive to amass. Furthermore, the *training* of such models is computationally demanding, and again is time-intensive. Without computational resources, one would be left to leverage cloud-computing to complete the training process, which is also an expensive prospect. In all, this remained beyond the computational-, time- and cost-capacity of this project.

The alternative method chosen, i.e. ablating configurations of utilizing pre-trained sentence embedding models, would be less successful by orders of magnitude. This is by some margin the most impactful obstruction to delivering on the goal of producing a ‘high quality representation’.

Therefore, future work would necessarily aim to develop a task-specific sentence embedding model that replaces those generic ones. This ‘pipeline’ of ablation experiments would again be trialled, perhaps with additional components or more expansive variations on those already listed (e.g. input-preprocessing or dimensionality reduction).

10.4.3 Baseline Model Design

Another crux of this study comes in the form of this baseline model that we use to evaluate representations. As demonstrated, this model does not claim to be indicative of *all* relevant semantic features, for this would bypass the need for any other representation method. This does not mean, however, that the baseline model is beyond reproach.

Firstly, the dictionary it foundationally derives from is subjective and incomplete. The bigger problem with this dictionary, however, is its need for manual curation - a part of the project’s motivation is to avoid such manual input, and instead rely on data-driven decisions as to what is, or isn’t, relevant skill or technical terminology.

Secondly, it could be criticized that the nature with which text passages are deemed similar according to this dictionary is not sufficient, since we are effectively only capturing token-level shared instances. We’re predicating that the context for the dictionary terms is as desired, without verification - e.g. if “Python and other snakes” is (for some reason) a sentence in a job posting text, then its baseline

measure is upvoted against Python-related repositories. To some extent we bypass this concern with our filtering of jobs by AI-relevant role titles, but the criticism is not without merit.

Another facet of what makes this baseline model problematic is the way in which we dictate the matching procedure - **lemmatization** is defensible, and widely used in NLP tasks, but **fuzzy-matching** is more ambiguously defined and as such is risky.

If a similar baseline model’s usage is necessary in future, further consideration must certainly be paid to ensure that its validity is more robust.

10.5 Opportunities for future work

Beyond altering the components that form this ‘pipeline’ as it exists, there are other elements of work which appear interesting and worthy of investigation.

First and foremost is the development of a task specific sentence transformer. Though not without cost, this would certainly yield the greatest benefits. This would offer *direct* application opportunities, since any party interested in understanding the dynamics of skill-related semantics on large text corpora can readily employ its functionality.

A parallel research question asks whether the clustering of GitHub repositories and job postings is helpful here. Predicated on a meaningful representation of both text documents, such clusters might yield highly interpretable insights. For example, they would group documents meaningfully and, in principle, one could assess their compositional evolution over time. From the GitHub repository side, clusters might change due to growing popularity of some repositories vs. others, or in the job posting side, clusters might change dependent on how language/semantics evolve with regard to a certain task/skill. BERTopic, a tool for interpretable clustering, would be a prime candidate for investigation in such pursuits.

And of course, on the topic of the original intention for this project, one could extend the similarity measures and evaluate their predictive power with regards to labour demand. Given that, there is some indication of lead-lag correlation for at least some repositories, this exploration could be expanded and developed further to aid policy-guiding forecasts. The benefits to such insights would be enormous, since they would inform more labour-demand forecasts more regularly and autonomously. Moreover, otherwise unaccounted for innovation signals existent in GitHub repositories would prove instrumental in explaining otherwise unanticipated shifts in demand.

Chapter 11

Conclusion

In summary, this paper investigates the relationship between GitHub repository and job posting texts, aiming to represent those skill/technology related features that are shared semantically. These representations thereafter undergo an ablation study, comparing relative their precision-based ranking performance against an evaluative baseline model, derived from a more primitive keyword-matching system. We compare the similarity-representations, using the temporal characteristic of the job postings, against the popularity time-series of repositories, providing a basis for investigating the nuanced relationship of software innovation and its adoption in labour demand.

Among the findings are that `bge-base-en-v1.5` appears to best represents the semantic setting we desire, and that input text has profound impacts on the performance of similarity ratings between sentence embeddings. From evaluating GitHub stars against the derived similarity time-series we find that there are themes of an explosion in AI-related repository popularities in around April 2023, with a surge in similarity results in around 2 – 3 quarters thereafter. In general, we find that repositories with increasing popularity often precede or coincide with heightened demand for the associated skills in job postings, suggesting that open-source activity may serve as a leading indicator of labour market trends. The results from this analysis are, however, hindered by several limitations: the reliance on a single embedding model, constraints on the size and representativeness of the job posting datasets, and the simplistic correlation analysis that does not fully account for confounding factors.

Future research could extend this pipeline by training bespoke sentence-models, exploring fine-tuning on domain-specific corpora, using clustering techniques to find informative skill- or repository-clusters, or implementing more sophisticated temporal forecasting techniques. Moreover, integrating additional innovation indicators (e.g., commit activity, forks, issues, or research publications) and expanding to multiple geographic or occupational contexts could provide a richer understand-

ing of the dynamics between technological innovation and labour demand.

In conclusion, this dissertation presents a modular, empirically validated pipeline for representing and comparing skills across job postings and open-source repositories, demonstrating both methodological advances and exploratory insights into the relationship between innovation and employment demand. By bridging these domains through semantic embeddings, the work contributes a foundation for future forecasting of skill demand informed by innovation signals.

References

- [1] N. Mankiw, *Macroeconomics* (Macroeconomics). Worth Publishers, 2000, ISBN: 978-1-57259-644-3. [Online]. Available: <https://books.google.co.uk/books?id=zb7QUr2S1ugC>.
- [2] A. G. Ross, P. G. McGregor, and J. K. Swales, “Labour market dynamics in the era of technological advancements: The system-wide impacts of labour augmenting technological change,” *Technology in Society*, vol. 77, p. 102 539, Jun. 2024, ISSN: 0160-791X. DOI: 10.1016/j.techsoc.2024.102539. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0160791X24000873> (visited on 08/19/2025).
- [3] *Luddite / Industrial Revolution, Machine-Breaking, Protest Movement / Britannica*, Jul. 2025. [Online]. Available: <https://www.britannica.com/event/Luddite> (visited on 08/19/2025).
- [4] A. Croitoru, “Schumpeter, Joseph Alois, 1939, Business Cycles: A Theoretical, Historical, and Statistical Analysis of the Capitalist Process,” *JOURNAL OF COMPARATIVE RESEARCH IN ANTHROPOLOGY AND SOCIOLOGY*, vol. 8, pp. 67–80, Aug. 2017.
- [5] B. Uzzi, S. Mukherjee, M. Stringer, and B. Jones, “Atypical combinations and scientific impact,” *Science (New York, N.Y.)*, vol. 342, no. 6157, pp. 468–472, Oct. 2013, ISSN: 1095-9203. DOI: 10.1126/science.1240474.
- [6] E. M. Rogers, *Diffusion of Innovations*, 3. ed. New York, NY: Free Press [u.a.], 1983, ISBN: 978-0-02-926650-2.
- [7] X. Wei, R. Liu, and W. Chen, “Meta theories of technological innovation based on the analysis of classic texts,” *Heliyon*, vol. 9, no. 6, e16779, Jun. 2023, ISSN: 2405-8440. DOI: 10.1016/j.heliyon.2023.e16779. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S2405844023039865> (visited on 08/19/2025).
- [8] J. Liu, H. Chang, J. Y.-L. Forrest, and B. Yang, “Influence of artificial intelligence on technological innovation: Evidence from the panel data of china’s manufacturing sectors,” *Technological Forecasting and Social Change*, vol. 158, p. 120 142,

- Sep. 2020, ISSN: 0040-1625. DOI: 10.1016/j.techfore.2020.120142. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0040162520309689> (visited on 08/19/2025).
- [9] J. Thomas, “Review of best practices in labour market forecasting with an application to the canadian aboriginal population,” *CSLS Research Reports*, 2015. [Online]. Available: <https://api.semanticscholar.org/CorpusID:155999286>.
 - [10] R. A. Wilson, “Forecasting skill requirements at national and company levels,” 2001. [Online]. Available: <https://api.semanticscholar.org/CorpusID:55484063>.
 - [11] D. M. Hausman and D. M. Hausman, “Why look under the hood?” In *The Philosophy of Economics: An Anthology*, Cambridge: Cambridge University Press, 2007, pp. 183–187.
 - [12] *Getting Skills Right: Skills for Jobs Indicators*. OECD, Jul. 2017, ISBN: 978-92-64-27787-8. DOI: 10.1787/9789264277878-en. [Online]. Available: <http://dx.doi.org/10.1787/9789264277878-en>.
 - [13] A. Parikh and R. Edwards, “An input-output approach to forecasting gross domestic output and employment intensities by sectors,” *Empirical Economics*, vol. 5, no. 1, pp. 1–14, Dec. 1980, ISSN: 1435-8921. DOI: 10.1007/BF01848039. [Online]. Available: <https://doi.org/10.1007/BF01848039>.
 - [14] R. A. Wilson, S.-A. Barnes, M. May-Gillings, P. Shyamoli, and H. Bui, *Working Futures 2017-2027 : Long-run labour market and skills projections for the UK : Main report*, London, Feb. 2020. [Online]. Available: <https://assets.publishing.service.gov.uk/government/uploads/system/uploads/attachment%5fdata/file/863506/Working%5fFutures%5fMain%5fReport.pdf>.
 - [15] *IER - Working Futures*. [Online]. Available: <https://warwick.ac.uk/fac/soc/ier/researchthemesoverview/researchprojects/wf/> (visited on 08/20/2025).
 - [16] *E3ME / Cambridge Econometrics*, Aug. 2024. [Online]. Available: <https://www.camecon.com/e3me> (visited on 08/20/2025).
 - [17] *Home - Office for National Statistics*. [Online]. Available: <https://www.ons.gov.uk/> (visited on 08/20/2025).
 - [18] *Labour market and skills projections: 2020 to 2035 - GOV.UK*. [Online]. Available: <https://www.gov.uk/government/publications/labour-market-and-skills-projections-2020-to-2035> (visited on 08/20/2025).
 - [19] C. Pullen, *Future skills projections and analysis : Research report : April 2024*, 2024. [Online]. Available: <https://dera.ioe.ac.uk/id/eprint/40664/>.

- [20] T. Dahlhaus, R. Ellwanger, G. Galassi, and P.-Y. Yanni, “From Online Job Postings to Economic Insights: A Machine Learning Approach to Structuring Naturally Occurring Data,” *AEA Papers and Proceedings*, vol. 115, pp. 73–78, May 2025, ISSN: 2574-0768. DOI: 10.1257/pandp.20251104. [Online]. Available: <https://www.aeaweb.org/articles?id=10.1257/pandp.20251104> (visited on 07/11/2025).
- [21] S. H. Bana, E. Brynjolfsson, D. Rock, and S. Steffen, “Job2vec : Learning a representation of jobs *,” 2021. [Online]. Available: <https://api.semanticscholar.org/CorpusID:236206510>.
- [22] *Using online job vacancies to understand the UK labour market from the bottom-up*, Jul. 2025. [Online]. Available: <https://www.bankofengland.co.uk/working-paper/2018/using-online-job-vacancies-to-understand-the-uk-labour-market-from-the-bottom-up> (visited on 08/20/2025).
- [23] *Skills in online job advertisements / CEDEFOP*, May 2017. [Online]. Available: <https://www.cedefop.europa.eu/en/projects/skills-online-job-advertisements> (visited on 08/20/2025).
- [24] “Demand for AI skills in jobs,” Organisation for Economic Co-Operation and Development (OECD), Tech. Rep., Mar. 2021.
- [25] M. Lukauskas, V. Šarkauskaitė, V. Pilinkienė, A. Stundžienė, A. Grybauskas, and J. Bruneckienė, “Enhancing Skills Demand Understanding through Job Ad Segmentation Using NLP and Clustering Techniques,” *Applied Sciences*, vol. 13, no. 10, p. 6119, Jan. 2023, ISSN: 2076-3417. DOI: 10.3390/app13106119. [Online]. Available: <https://www.mdpi.com/2076-3417/13/10/6119> (visited on 07/09/2025).
- [26] D. Deming and L. B. Kahn, “Skill Requirements across Firms and Labor Markets: Evidence from Job Postings for Professionals,” *Journal of Labor Economics*, vol. 36, no. S1, S337–S369, Jan. 2018, ISSN: 0734-306X. DOI: 10.1086/694106. [Online]. Available: <https://www.journals.uchicago.edu/doi/abs/10.1086/694106> (visited on 08/18/2025).
- [27] E. M. Sibarani, S. Scerri, C. Morales, S. Auer, and D. Collarana, “Ontology-guided Job Market Demand Analysis: A Cross-Sectional Study for the Data Science field,” in *Proceedings of the 13th International Conference on Semantic Systems*, ser. Semantics2017, New York, NY, USA: Association for Computing Machinery, Sep. 2017, pp. 25–32, ISBN: 978-1-4503-5296-3. DOI: 10.1145/3132218.3132228. [Online]. Available: <https://dl.acm.org/doi/10.1145/3132218.3132228> (visited on 07/09/2025).

- [28] “Open-source software,” *Wikipedia*, Aug. 2025. [Online]. Available: https://en.wikipedia.org/w/index.php?title=Open-source_software&oldid=1306210705 (visited on 08/20/2025).
- [29] “GitHub,” *Wikipedia*, Aug. 2025. [Online]. Available: <https://en.wikipedia.org/w/index.php?title=GitHub&oldid=1305898104> (visited on 08/20/2025).
- [30] *Roads and Bridges: The Unseen Labor Behind Our Digital Infrastructure*, Jul. 2016. [Online]. Available: <https://www.fordfoundation.org/work/learning/research-reports/roads-and-bridges-the-unseen-labor-behind-our-digital-infrastructure/> (visited on 08/18/2025).
- [31] *Why open source is critical to the future of AI*. [Online]. Available: <https://www.redhat.com/en/blog/why-open-source-critical-future-ai> (visited on 08/20/2025).
- [32] *Open source: Why open ecosystems matter*, Jul. 2025. [Online]. Available: <https://www.itpro.com/technology/artificial-intelligence/open-source-why-open-ecosystems-matter> (visited on 08/20/2025).
- [33] H. Fang, P. Park, J. Evans, J. Herbsleb, and B. Vasilescu, *Weak Ties Explain Open Source Innovation*, Nov. 2024. DOI: 10.48550/arXiv.2411.05646. arXiv: 2411.05646 [cs]. [Online]. Available: <http://arxiv.org/abs/2411.05646> (visited on 08/18/2025).
- [34] G. Mészáros and J. Wachs, *The Dynamics of Innovation in Open Source Software Ecosystems*, Nov. 2024. DOI: 10.48550/arXiv.2411.14894. arXiv: 2411.14894 [cs]. [Online]. Available: <http://arxiv.org/abs/2411.14894> (visited on 08/18/2025).
- [35] E. M. Brown, C. Osborne, P. Cihon, *et al.*, *Measuring Software Innovation with Open Source Software Development Data*, Jun. 2025. DOI: 10.48550/arXiv.2411.05087. arXiv: 2411.05087 [cs]. [Online]. Available: <http://arxiv.org/abs/2411.05087> (visited on 08/20/2025).
- [36] H. Borges and M. Tulio Valente, “What’s in a GitHub Star? Understanding Repository Starring Practices in a Social Coding Platform,” *Journal of Systems and Software*, vol. 146, pp. 112–129, Dec. 2018, ISSN: 0164-1212. DOI: 10.1016/j.jss.2018.09.016. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0164121218301961> (visited on 07/16/2025).
- [37] H. Borges, A. Hora, and M. T. Valente, “Understanding the Factors that Impact the Popularity of GitHub Repositories,” in *2016 IEEE International Conference on Software Maintenance and Evolution (ICSME)*, Oct. 2016, pp. 334–344. DOI: 10.1109/ICSME.2016.31. arXiv: 1606.04984 [cs]. [Online]. Available: <http://arxiv.org/abs/1606.04984> (visited on 08/18/2025).

- [38] S. Koch, D. Klein, and M. Johns, “The Fault in Our Stars: An Analysis of GitHub Stars as an Importance Metric for Web Source Code,” *Proceedings 2024 Workshop on Measurements, Attacks, and Defenses for the Web*, 2024. DOI: 10.14722/madweb.2024.23004. [Online]. Available: <https://www.ndss-symposium.org/wp-content/uploads/madweb2024-4-paper.pdf> (visited on 08/20/2025).
- [39] H. Fang, J. Herbsleb, and B. Vasilescu, “Novelty begets long-term popularity, but curbs participation: A macroscopic view of the python open-source ecosystem,” in *International Conference on Software Engineering*, ser. ICSE, IEEE, 2024. DOI: 10.1145/3597503.3608142.
- [40] M. A. Moid, A. Siraj, M. F. Ali, and A. O. Amoodi, “Predicting stars on open-source GitHub projects,” in *2021 Smart Technologies, Communication and Robotics (STCR)*, 2021, pp. 1–9. DOI: 10.1109/STCR51658.2021.9588891.
- [41] T. Mikolov, K. Chen, G. Corrado, and J. Dean, *Efficient Estimation of Word Representations in Vector Space*, Sep. 2013. DOI: 10.48550/arXiv.1301.3781. arXiv: 1301.3781 [cs]. [Online]. Available: <http://arxiv.org/abs/1301.3781> (visited on 08/20/2025).
- [42] J. Pennington, R. Socher, and C. Manning, “Glove: Global Vectors for Word Representation,” in *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, Doha, Qatar: Association for Computational Linguistics, 2014, pp. 1532–1543. DOI: 10.3115/v1/D14-1162. [Online]. Available: <http://aclweb.org/anthology/D14-1162> (visited on 08/20/2025).
- [43] A. Vaswani, N. Shazeer, N. Parmar, *et al.*, “Attention is all you need,” in *Advances in Neural Information Processing Systems*, I. Guyon, U. V. Luxburg, S. Bengio, *et al.*, Eds., vol. 30, Curran Associates, Inc., 2017. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf.
- [44] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*, May 2019. DOI: 10.48550/arXiv.1810.04805. arXiv: 1810.04805 [cs]. [Online]. Available: <http://arxiv.org/abs/1810.04805> (visited on 08/20/2025).
- [45] N. Reimers and I. Gurevych, *Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks*, Aug. 2019. DOI: 10.48550/arXiv.1908.10084. arXiv: 1908.10084 [cs]. [Online]. Available: <http://arxiv.org/abs/1908.10084> (visited on 08/18/2025).
- [46] *The ESCO Classification*. [Online]. Available: <https://esco.ec.europa.eu/en/classification> (visited on 08/18/2025).

- [47] C. Sleeman, *An Open and Data-driven Taxonomy of Skills Extracted from Online Job Adverts (ESCoE DP 2018-13)*, Aug. 2018. [Online]. Available: <https://www.escoe.ac.uk/publications/an-open-and-data-driven-taxonomy-of-skills-extracted-from-online-job-adverts/> (visited on 08/18/2025).
- [48] M. Zhang, K. N. Jensen, S. Sonniks, and B. Plank, “SkillSpan: Hard and soft skill extraction from English job postings,” in *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, Seattle, United States: Association for Computational Linguistics, Jul. 2022, pp. 4962–4984. [Online]. Available: <https://aclanthology.org/2022.naacl-main.366>.
- [49] J. Lafferty, A. McCallum, F. Pereira, and H. Street, “Conditional Random Fields: Probabilistic Models for Segmenting and Labeling Sequence Data,”
- [50] M. O. F. Rokon, P. Yan, R. Islam, and M. Faloutsos, *Repo2Vec: A Comprehensive Embedding Approach for Determining Repository Similarity*, Jul. 2021. DOI: 10.48550/arXiv.2107.05112. arXiv: 2107.05112 [cs]. [Online]. Available: <http://arxiv.org/abs/2107.05112> (visited on 08/20/2025).
- [51] M. Izadi, A. Heydarnoori, and G. Gousios, *Topic Recommendation for Software Repositories using Multi-label Classification Algorithms*, Jun. 2021. DOI: 10.48550/arXiv.2010.09116. arXiv: 2010.09116 [cs]. [Online]. Available: <http://arxiv.org/abs/2010.09116> (visited on 08/20/2025).
- [52] H. Shao, D. Sun, J. Wu, *et al.*, “Paper2repo: GitHub Repository Recommendation for Academic Papers,” in *Proceedings of The Web Conference 2020*, Apr. 2020, pp. 629–639. DOI: 10.1145/3366423.3380145. arXiv: 2004.06059 [cs]. [Online]. Available: <http://arxiv.org/abs/2004.06059> (visited on 08/20/2025).
- [53] Y. Zhang, F. F. Xu, S. Li, *et al.*, “HiGitClass: Keyword-driven hierarchical classification of GitHub repositories,” in *2019 IEEE International Conference on Data Mining (ICDM)*, Los Alamitos, CA, USA: IEEE Computer Society, Nov. 2019, pp. 876–885. DOI: 10.1109/ICDM.2019.00098. [Online]. Available: <https://doi.ieeeecomputersociety.org/10.1109/ICDM.2019.00098>.
- [54] M. U. Mehmood, S. Hussain, W. L. Wang, and M. U. Malik, *LLM-based Content Classification Approach for GitHub Repositories by the README Files*, Jul. 2025. DOI: 10.48550/arXiv.2507.21899. arXiv: 2507.21899 [cs]. [Online]. Available: <http://arxiv.org/abs/2507.21899> (visited on 08/20/2025).
- [55] LinkedIn Corporation, *LinkedIn website*. [Online]. Available: <https://www.linkedin.com/> (visited on 08/21/2025).
- [56] Indeed Inc., *Indeed website*. [Online]. Available: <https://www.indeed.com/> (visited on 08/21/2025).

- [57] Lightcast, *Lightcast - A Global Leader in Labour Market Analytics*. [Online]. Available: <https://lightcast.io/uk> (visited on 08/21/2025).
- [58] *Find Open Datasets and Machine Learning Projects / Kaggle*. [Online]. Available: <https://www.kaggle.com/datasets> (visited on 08/21/2025).
- [59] *Machine Learning Job Postings in the US*. [Online]. Available: <https://www.kaggle.com/datasets/ivankmk/thousand-ml-jobs-in-usa> (visited on 07/16/2025).
- [60] Asaniczka, *Linkedin data engineer job postings*, 2023. DOI: 10.34740/KAGGLE/DSV/7292935. [Online]. Available: <https://www.kaggle.com/dsv/7292935>.
- [61] Asaniczka, *Data science job postings & skills (2024)*, 2024. DOI: 10.34740/KAGGLE/DS/4407481. [Online]. Available: <https://www.kaggle.com/ds/4407481>.
- [62] A. Koneru, *LinkedIn job postings (2023 - 2024)*, 2024. DOI: 10.34740/KAGGLE/DSV/9200871. [Online]. Available: <https://www.kaggle.com/dsv/9200871>.
- [63] *Job Postings from Ireland (October 2022)*. [Online]. Available: <https://www.kaggle.com/datasets/techmap/job-postings-ireland-october-2022> (visited on 07/16/2025).
- [64] *US Job Postings from 2023-05-05*. [Online]. Available: <https://www.kaggle.com/datasets/techmap/us-job-postings-from-2023-05-05> (visited on 07/16/2025).
- [65] *Overview on Techmap's Job Data & Data Samples*. [Online]. Available: <https://jobdatafeeds.com/job-data-overview> (visited on 08/21/2025).
- [66] *OSS Insight*. [Online]. Available: <https://ossinsight.io/> (visited on 08/21/2025).
- [67] *We are going to redesign the trending algorithm · Issue #778 · pingcap/ossinsight*. [Online]. Available: <https://github.com/pingcap/ossinsight/issues/778> (visited on 08/21/2025).
- [68] N. Reimers and I. Gurevych, *Sentence-BERT: Sentence embeddings using siamese BERT-networks*, 2019. [Online]. Available: <https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2>.
- [69] F. Schroff, D. Kalenichenko, and J. Philbin, "FaceNet: A Unified Embedding for Face Recognition and Clustering," in *2015 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, Jun. 2015, pp. 815–823. DOI: 10.1109/CVPR.2015.7298682. arXiv: 1503.03832 [cs]. [Online]. Available: <http://arxiv.org/abs/1503.03832> (visited on 09/02/2025).
- [70] T. Fawcett, "An introduction to ROC analysis," *Pattern Recognition Letters*, vol. 27, no. 8, pp. 861–874, 2006.

- [71] W. Wang, F. Wei, L. Dong, H. Bao, N. Yang, and M. Zhou, *MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers*, 2020. arXiv: 2002.10957 [cs.CL].
- [72] Sentence-Transformers, *All-mpnet-base-v2: Sentence embedding model*, 2025. [Online]. Available: <https://huggingface.co/sentence-transformers/all-mpnet-base-v2>.
- [73] K. Song, X. Tan, T. Qin, J. Lu, and T.-Y. Liu, *MPNet: Sentence embedding model*, 2020. [Online]. Available: <https://arxiv.org/abs/2004.09297>.
- [74] K. Song, X. Tan, T. Qin, J. Lu, and T.-Y. Liu, *MPNet: Masked and Permuted Pre-training for Language Understanding*, Nov. 2020. DOI: 10.48550/arXiv.2004.09297. arXiv: 2004.09297 [cs]. [Online]. Available: <http://arxiv.org/abs/2004.09297> (visited on 09/02/2025).
- [75] B. R. Team, *BGE: BERT-based general embedding for dense retrieval*, 2023. [Online]. Available: <https://huggingface.co/BAAI/bge-base-en-v1.5>.
- [76] N. Muennighoff, N. Tazi, L. Magne, and N. Reimers, *MTEB: Massive Text Embedding Benchmark*, Mar. 2023. DOI: 10.48550/arXiv.2210.07316. arXiv: 2210.07316 [cs]. [Online]. Available: <http://arxiv.org/abs/2210.07316> (visited on 09/04/2025).
- [77] Cursor, *Cursor AI code editor*, 2025. [Online]. Available: <https://cursor.sh/>.
- [78] OpenAI, *ChatGPT*, 2025. [Online]. Available: <https://chat.openai.com/>.
- [79] Elicit, *Elicit: The AI research assistant*, 2023-01-24, 2023. [Online]. Available: <https://elicit.com>.
- [80] *About the GraphQL API*. [Online]. Available: <https://docs-internal.github.com/en/graphql/overview/about-the-graphql-api> (visited on 09/04/2025).

Appendix A

A.1 Generative AI

This paper uses 3 Generative AI Tools:

1. **Cursor IDE** for expanding the weakly supervised model keyword dictionary, and implementing code efficiency improvements (using GPT-5) [77], [78]
2. **Elicit AI** to assist in literature exploration phase [79]
3. **ChatGPT** for advice, critique & education [78]

None of these tools were used to generate, or otherwise automate, the content of this paper (Except in the specific case of A.1.3.2) Underlying my use of generative AI was the imperative to avoid “Copy & Paste” strategies and instead use them as research, exploration and educational tools.

A.1.1 Elicit

For research, Elicit is a useful tool that performs a complex literature search based on detailed prompts. This was valuable for exploring research & informing the thesis more precisely.

I would, in future, make greater use of this tool for guiding the literature review stage.

Here’s are the *exact* prompts:

Prompt: Job Description Research

To what extent can we use the “Job description” section of Job Advertisements to map, forecast or understand the demand of certain skills or job requirements? A methodology that describes time-series representation of features in the Natural Language text, with NLP or other implementations.

Prompt: Innovation informed AI-Skill Forecast Research

Can innovation signal data (research papers, patents or open source software (e.g. GitHub)) improve the strength of skill demand forecasts derived from job postings (job boards like LinkedIn or Indeed), particularly for AI related skills? E.g. “prompt engineering” or “MLOps”

A.1.2 ChatGPT

For Code, ChatGPT was used for advice, debugging code or learning new programming frameworks. For writing, it was used as a critique assistant, to help explore ideas and critically assess the coherence of arguments or passages.

Here are some example prompts (adjusted, on a case-by-case basis):

Prompt: Conceptual Understanding

*Advise me on how to think about **{Concept/Tool/Framework/Method}** conceptually. I'd like to know the mental models or guiding intuitions that professionals use.*

Prompt: Conceptual Understanding

*Below is my summary of what I currently understand about **{Concept/-Tool/Framework/Method}**. Please critique it: point out what is accurate, what is incomplete, and where I've misunderstood. The goal is to refine my own understanding, not to receive a polished replacement text.*

Much of the time I use ChatGPT for workflow and tool advice, especially with regards to using Latex:

Prompt: Workflow advice

*I'm setting up a research workflow using **{Tool/Method}**. Can you suggest general strategies, potential pitfalls, and best practices? My goal is to design the workflow myself, but I'd like conceptual guidance.*

Prompt: L^AT_EXHelp

Advise me on strategies for managing a large dissertation in LaTeX. For example, what are best practices for splitting chapters into separate files, managing bibliographies, and handling figures/tables so that the project remains maintainable?

Prompt: Code Critique

*Below I include a piece of code. Rather than rewriting it for me, can you summarize your understanding of its purpose, critique its efficiency, point out any readability/-maintainability issues, and suggest what **principles** I should look at to improve it?*
{Code}

Prompt: Writing Critique

*I am writing a dissertation in **{Dissertation Context}**. I will paste a passage of my draft below. Please provide a detailed critique focusing on:*

- 1. clarity of expression and readability,*
- 2. academic style and tone,*
- 3. logical flow and argument structure, and*
- 4. any redundancy, ambiguity, or opportunities for conciseness.*

*Give only high-level feedback (structure, argument, coherence) - do not provide any specific rephrasing or line edits. Assume I am aiming for a professional, dissertation-quality standard. Here is the passage: **{Text Passage (No more than a paragraph at a time)}***

Each of the above prompts are used as guiding structures, and much of the time were adapted in scenario-specific follow-ups.

A.1.3 Cursor

A.1.3.1 Code Improvements

Cursor was used to advise on code improvements that were made strictly *after* a working solution was in place. The purpose of this was to improve both the computational efficiency and readability of the codebase. Further, it serves as an educational tool for improving my coding skills.

These adjustments were not made using the “Agent” feature of Cursor, but with “Ask”, such that it provides its suggestion for implementation at my discretion.

Here’s an example prompt for this use-case:

Prompt: Code Improvement Suggestions

*Within **{Context File}** in lines **{Context Lines}** is a piece of code. Currently it is functional. Rather than rewrite it for me, can you summarize your understanding of its purpose, critique its efficiency, point out any readability/maintainability issues, and suggest what **principles** I should look at to improve it?*

A.1.3.2 Baseline Model Keyword Generation

This is sole exception case where generated content was used: Instructing Cursor (using GPT-5) to construct the keyword dictionary for the weakly-supervised baseline model. Its task here was to (In iterations of 50 items; 4 runs → 200 items):

- Expand a *Seed List* of keywords (Obtained by iteratively scraping GitHub topics)
- Match keywords to their commonly used acronyms
- Categorize the keywords

The grounding of this implementation in a *Seed List* was **instrumental** in making this effort defensible.

The purpose of this was to reduce manual & laborious data entry work - it should be noted that the only practical advantage of this implementation was to save time and effort. This entire process was audited and refined manually thereafter, to ensure output quality.

Prompt: Keyword List Curation

In {Context File} you'll find a list of keywords relating to Artificial Intelligence & related GitHub repository topics. Your task is to expand this keyword list, to a total of no more than 50. The resulting list should contain core concepts, tools, programming languages and domain-specific terms. If a keyword is commonly referred to by an acronym, these should be paired with its relevant verbose version. The acronym field should be left null for terms without commonly used acronyms. Be sure to preserve the casing for each keyword - e.g. "pytorch" should be "PyTorch". Make sure not to duplicate existing entries.

Prompt: Categorization

Within {Context File} are keywords that relate to artificial intelligence (including their acronyms). Create a new field of categories for each keyword into a set of no more than 25 different categories. These categories should be informative, and concise; if deemed necessary, one term can belong to multiple categories (no more than 3).

Data Sources

Table 4.1 describes the data used for job postings.

For GitHub repositories, data was derived from GitHub via its GraphQL API endpoint [80].

Auxiliary data used includes those names of relevant GitHub repositories, obtained from *OSS Insight*[66].

Third Party Code and Software Libraries

The Python programming language was used for this project. All associated libraries are listed in the enpackaged `requirements.txt`. Key libraries include: `numpy`, `pandas`, `scikit-learn`, `matplotlib`, `requests`, `sentence_transformers`